DEPARTMENT OF SOFTWARE ENGINEERING
SABARAGAMUWA UNIVERSITY OF SRI LANKA

SE8101 ABSTRACT BOOK

SE 2020/2021 BATCH

August 8, 2026

Sabaragamuwa University of Sri Lanka
Belihuloya, Sri Lanka

Table of Contents

Page Number

Cross-Project Explainable Defect Prediction with Human-Centric Explanations

[1]R.M.J.M. Rathnayaka* and [1]H.M.C. Nirmani
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*rmjmrathnayaka@std.appsc.sab.ac.lk

Cross-Project Defect Prediction (CPDP) supports software projects that lack sufficient labelled history for reliable within-project models. However, differences among repositories and limited interpretability reduce practical adoption. This study developed an explainable CPDP framework for Python repositories by combining static code metrics, process metrics, Logistic Regression, exact Linear SHAP explanations, human-centric rules, and a Streamlit repository-analysis application. The dataset contained 7,327 file observations from 27 publicly available Python repositories. The extracted records were divided into CSV files containing a maximum of 500 records and labelled by industry experts. The final dataset contained 5,550 positive-labelled and 1,777 negative-labelled observations. Five static metrics and three process metrics were evaluated using Leave-One-Project-Out testing. Logistic Regression, Random Forest, and XGBoost were compared across static-only, process-only, and combined feature groups. Process-only XGBoost achieved the highest initial macro Matthews Correlation Coefficient (MCC) of 0.5410. Combined Logistic Regression achieved a close MCC of 0.5317 and was selected for the final human-centric framework because it provided stronger balanced accuracy, specificity, precision, transparency, and coverage of both static and process features. A second experiment compared no adaptation and CORAL adaptation with no imbalance handling, class weighting, and SMOTE. Logistic Regression with SMOTE and no CORAL achieved the best macro MCC of 0.5321, balanced accuracy of 0.8210, F1-score of 0.8934, ROC-AUC of 0.9288, and PR-AUC of 0.9672. Linear SHAP ranked number of developers, commit frequency, and average function length as the most influential features. The explanations reconstructed model outputs with negligible numerical error and showed strong ranking stability across five SMOTE seeds. The Streamlit prototype accepts a public GitHub repository, extracts compatible metrics, ranks files by review-priority score, displays increasing and reducing contributions, provides developer-oriented actions, and exports CSV and HTML reports.

Keywords: *Cross-project defect prediction, Human-centric explanations, Linear SHAP, Logistic regression, Software metrics*

Cross-Language Game Engine Extensions: A Rust-Powered Performance Layer for Unity using C# Interoperability

[1]W.K.W. Samarasinghe* and [2]L.S. Lekamge
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Sabaragamuwa University of Sri Lanka
*wkwsamarasingha@std.appsc.sab.ac.lk

Unity primarily uses managed C# for gameplay development, but computationally intensive workloads may benefit from optimised or native execution. This study designed a Rust- powered native performance layer for Unity using a C-compatible Foreign Function Interface (FFI) and C# P/Invoke. The approach was compared with standard C#, Unity Burst, and a conventional C++ native plug-in. Equivalent implementations of A* pathfinding, Boids simulation, procedural heightmap generation, and FFI microbenchmarks were evaluated using Windows x86-64 standalone Unity players. The evaluation combined correctness validation, repeated performance measurements, runtime profiling, source-code analysis, native safety testing, a researcher retrospective, and an exploratory participant questionnaire. Across the tested workload sizes, Rust achieved mean speedups of approximately 2.02× for A*, 7.33× for Boids, and 10.19× for heightmap generation compared with standard C#. Burst was the fastest A* approach, while Rust slightly exceeded the single-threaded Burst implementations for Boids and heightmap generation and remained close to C++. The minimal Rust FFI call introduced an empirical fixed overhead of approximately 2.73 ns, but larger computations compensated for this cost. Runtime profiling recorded zero managed allocations per operation and no generation-0, generation-1, or generation-2 garbage collections during the final profiling intervals. Rust localised explicit unsafe operations mainly to the interoperability boundary, while the principal kernels remained in safe Rust. All 86 non-destructive invalid-input tests passed; however, 71 of 78 isolated destructive observations terminated the child process, showing that neither Rust nor C++ can protect the complete system when the caller violates pointer or lifetime contracts. The findings show that Rust is a feasible, high-performing option for sufficiently large and self-contained Unity workloads. Its benefits are strongest when computation is batched into coarse-grained native calls. Unity Burst remains preferable when it supports the workload with less integration effort, whereas every native design requires strict ABI, validation, ownership, and resource lifetime controls.

Keywords: *Foreign function interface, Memory safety, Performance benchmarking, Unity, Rust*

Latency and Performance of Cloud vs Edge Deployment in Kubernetes Environments: An Empirical Study

[1]D.G.C. Shehani* and [2]K.P.N. Jayasena
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Researcher, Faculty of Computer Science, Technical University Dresden, Germany
*dgcshehani@std.appsc.sab.ac.lk

The growing deployment of latency-sensitive applications in healthcare, automotive, and industrial sectors has intensified demand for evidence-based guidance on choosing between cloud and edge computing architectures. While theoretical and simulation-based studies widely predict a latency advantage for edge deployment, reproducible empirical comparisons using real Kubernetes infrastructure under systematic network degradation conditions remain scarce. This study designs and implements a real two-virtual-machine KubeEdge-based Kubernetes experimental environment — VM1 running Minikube, CloudCore, and Istio IngressGateway; VM2 running KubeEdge EdgeCore and containerd — and empirically measures End-to-End (E2E), Pod-to-Pod (P2P), and Ingress latency for both Edge and Cloud deployments across a 4×3 matrix of 12 network degradation scenarios combining four delay levels (10ms, 50ms, 100ms, 200ms) with three packet loss levels (0%, 5%, 10%). Results from 72 experimental measurements reveal that packet loss rate, not network delay, is the primary determinant of architecture selection. Under zero packet loss, Cloud achieves lower p95 latency at most delay levels, with a near-deterministic standard deviation as low as 3.32ms at 100ms/0%. Under 5% packet loss, Edge is strongly preferred at all tested delay levels: at 50ms/5%, Edge p95 of 87ms is 218ms lower than Cloud p95 of 305ms — an 11.7-fold difference in sensitivity — while at 200ms/5%, Edge p95 of 256ms is 350ms lower than Cloud p95 of 606ms. At 10% packet loss, both architectures converge to within 5ms at every delay level. Cloud also produces catastrophic tail latency spikes under packet loss — reaching 7,534ms maximum at 100ms/5% — versus Edge's maximum of 721ms in the same scenario. Edge achieves 30.9% higher throughput at 100ms/5% despite similar p95 values, driven by Cloud's extreme tail requests occupying virtual users. The study also corrects a methodological error in Pod-to-Pod measurement found in the original scripts and establishes that the KubeEdge architecture introduces 3–11ms P2P overhead for the edge path. The experimental framework is made publicly available as a reproducible GitHub repository, addressing the reproducibility gap identified in existing literature.

Keywords: *Cloud computing, Edge computing, KubeEdge, Kubernetes, Latency*

Augmenting LLM-Based Multi-Agent Software Quality Assurance with Specialised Machine-Learning Microservices

[1]M.P.S. Alwis* and [2]R.M.K.K. Wijerathna
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*mmpsalvis@std.appsc.sab.ac.lk

This research investigates an ML-augmented LLM-based multi-agent software quality assurance framework designed to address cold-start test planning, brittle UI selectors, and passive security blindness in CI/CD pipelines. The proposed system coordinates specialised QA agents with independently versioned machine-learning microservices for defect-risk prioritisation, UI selector repair, and HTTP traffic anomaly detection. Using a design-science methodology and controlled evaluation protocols, the study compares a baseline LLM-only multi-agent workflow with augmented configurations using defect yield per token, cross- language AUC-ROC, selector repair rate, security recall at a constrained false-positive rate, and runtime overhead. The research contributes a reproducible architecture that connects multi-agent orchestration with lightweight ML services to improve the measurability, efficiency, and operational reliability of automated software quality assurance.

Keywords: *Defect prediction, Large language models, Multi-Agent systems, Security anomaly detection, Selector repair*

Explainable AI for Real-Time Performance Anomaly Detection in Progressive Web Application

[1]K.A.P.I Fernando* and [2]W.T.S. Somaweera
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*kapifernando@std.appsc.sab.ac.lk

Progressive Web Applications provide app-like web experiences, but their performance can vary because of browser behaviour, device resources, rendering activity, memory use, network conditions, and user interactions. Traditional threshold-based monitoring can raise alerts without clearly explaining why a problem occurred. This study developed an explainable anomaly detection prototype that combines an LSTM autoencoder with Kernel SHAP. A single labelled master dataset containing 17,539 observations was analysed, including 10,944 normal and 6,595 anomalous records. Five client-side measurements were used: CPU time, JavaScript heap usage, Largest Contentful Paint, Cumulative Layout Shift delta, and Interaction to Next Paint. Normal observations were divided chronologically into training, validation, and test partitions. A MinMaxScaler was fitted only on the normal training partition, and overlapping sequences of ten-time steps were created. The autoencoder was trained only on normal sequences, and reconstruction Mean Absolute Error was used as the anomaly score. On the combined test set of 8,219 sequences, the selected threshold produced 90.50% accuracy, 99.49% precision, 88.60% recall, 98.16% specificity, an F1- score of 93.73%, and a ROC AUC of 0.9779. The confusion matrix contained 1,603 true negatives, 30 false positives, 751 false negatives, and 5,835 true positives. A Kernel SHAP explanation for one detected anomaly identified CLS Delta as the largest contributor, followed by JavaScript heap usage and LCP. The results show that temporal reconstruction models can separate the controlled normal and anomaly patterns well and can provide understandable feature-level evidence. The findings demonstrate that the proposed approach can help developers identify abnormal PWA behaviour and understand which performance metrics are mainly responsible for each anomaly. This can reduce manual investigation time, support faster root-cause analysis, and help development teams make more informed performance-optimization decisions. The approach therefore provides a practical basis for building intelligent and developer-friendly monitoring tools for Progressive Web Applications.

Keywords: *Anomaly detection, Explainable AI, LSTM autoencoder, Progressive web applications, SHAP*

Explainable Software Defect Root Cause Analysis using Commit History and Developer Activity Metrics

[1]R.G.R.L. Ranathunga* and [2]L.S. Lekamge
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing & Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*rgrlranathunga@std.appsc.sab.ac.lk

Just-In-Time (JIT) defect prediction identifies risky commits before defects propagate, but prediction-only systems provide limited guidance about why a change is flagged. This research develops and evaluates an explainable defect risk analysis framework using commit history and developer activity metrics. ApacheJIT commits from 2015 onward were analysed. The official modern training set contained 14,616 commits, while a commit-disjoint held-out pool contained 39,925 commits and a balanced test benchmark contained 11,636 commits. Twelve original metrics were expanded into 32 predictive features, including a leakage-controlled past-only project defect prior. Logistic Regression with Bonferroni correction, variance inflation factors, and bootstrap stability identified lines added, lines deleted, subsystem count, change entropy, file age, and recent author experience as statistically significant. Random Forest, XGBoost, LightGBM, and CatBoost were tuned using temporal cross-validation and evaluated alongside calibrated logistic stacking. LightGBM achieved the highest balanced-test F1-score of 80.90%, with 80.72% accuracy, 81.69% recall, 89.06% ROC-AUC, and 87.72% PR-AUC. CatBoost was selected by validation F1 for SHAP-based explanation and single-model deployment. SHAP provided stable and directly model-based explanations, while LIME showed lower local fidelity. Explanations were translated into natural-language risk factors, protective factors, dual-evidence findings, and review actions. Leave-one-project-out evaluation achieved macro ROC-AUC of 0.8079 and macro PR-AUC of 0.7936. A paired evaluation with 25 developers across 125 scenarios showed significant improvements in understanding, trust, usefulness, and actionability for explanation-supported output (Wilcoxon $p < 0.001$). The implemented Python package supports metric extraction, scoring, command-line and HTTP interfaces, and CI/CD integration.

Keywords: *Calibrated stacking, Developer evaluation, Explainable artificial intelligence, Just-In-Time defect prediction, Software quality*

Lightweight edge ai frame work for predictive auto-scaling of containerized microservices

[1]H.M.T.N. Padiwela* and [2] S. Vasanthapriyan
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*hmtnpadiwela@std.appsc.sab.ac.lk

This paper studies the current trends of microservices containerization and the associated difficulties in ensuring reliable operation amidst volatile changes in workload. The existing reactive auto-scaling techniques often display the tendency towards slow responsiveness when faced with the unexpected increase in traffic volume, leading to the exhaustion of resources and poor performance. On the other hand, predictive auto-scaling solutions available to date are usually based on bulky machine learning models run by a central processing server. As a way of overcoming these limitations, this paper offers a light and decentralized approach to predictive auto-scaling via the deployment of a tiny TensorFlow Lite (TFLite) model in proximity to application containers on the edge. Due to the low number of instances of extreme loads reflected in the cloud telemetry datasets available, the paper suggests developing a synthetic data augmentation process using the trace of Google Cloud workloads. By combining the information about normal traffic flow and stressful situations, the paper aims to enable the model to predict the potential exhaustion of resources in the nearest future. The development and evaluation of the proposed solution took place in an Amazon Web Services (AWS) environment comprising Docker microservices, Prometheus monitoring, and cAdvisor telemetry system. Based on the experiments conducted with high-traffic workloads, it has been found out that the proposed 600 KB TFLite model provides 92% anomaly recall rate and RMSE decrease from 0.12 to 0.08 in comparison with standard training on the raw datasets. Besides, the tiny model allows making quick scaling decisions with the minimum use of memory resources, proving the feasibility of deploying smart auto-scaling systems without expensive centralized artificial intelligence facilities.

Keywords: *Containerized microservices, Edge AI, Predictive auto-scaling, Synthetic telemetry, TensorFlow lite*

An Empirical Evaluation of CI/CD Automation and Cloud-Native DevOps Frameworks in Optimizing the Software Development Lifecycle

[1]M.J.A. Ahamed* and [2]L.S. Lekamge
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing & Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*mjaahamed@std.appsc.sab.ac.lk

Selecting suitable Continuous Integration and Continuous Delivery (CI/CD) automation tools and cloud-native DevOps frameworks is an important architectural decision, as it can directly influence deployment speed, system reliability, and long-term sustainability. Although DevOps has been widely discussed in industry and academic literature, there is still a lack of controlled, empirical, and reproducible comparisons, particularly for small-to-medium teams working with limited resources. This research addresses this gap through a seven-phase empirical methodology that compares two CI/CD pipeline architectures using the same three-tier web application, TaskFlow (React, Node.js/Express, and PostgreSQL). Pipeline A used Jenkins with Docker Compose, while Pipeline B used GitHub Actions with Kubernetes. A total of 56 controlled pipeline runs, 28 for each architecture, were conducted across 25 scenario scripts, with performance measured using DORA metrics. The results showed a significant difference in lead time, with Pipeline A averaging 2.24 minutes compared with 6.26 minutes for Pipeline B, ... giving Pipeline A a 2.8× advantage ($p < 0.001$, $t = -96.37$, $df = 40$). Pipeline A achieved a raw deployment success rate of 78.6%, compared with 71.4% for Pipeline B; after excluding initial setup failures, these increased to 92.9% and 85.7%, respectively. Both pipelines detected all four intentional defects with 100% accuracy. However, Pipeline A required more setup effort, involving 12 issues and 260 minutes of troubleshooting, compared with 5 issues and 130 minutes for Pipeline B, including a recurring Docker socket problem. Pipeline B also provided automatic trigger-on-push functionality. Overall, neither architecture was universally better: Jenkins with Docker Compose suited small, co-located academic teams, while GitHub Actions with Kubernetes was more appropriate for growth-stage, production-focused teams requiring automatic triggers, distributed access, and scalability.

Keywords: *CI/CD pipelines, DevOps, GitHub actions, Jenkins, Kubernetes*

REMARL: A Reinforcement Learning Based Approach for Multi-Agent Requirements Engineering

[1]M.Z.I.A.A.S. Ibrahim*, [2]B.T.G.S. Kumara
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*mziaasibrahim@std.appsc.sab.ac.lk

Requirements Engineering (RE) determines whether software projects succeed or fail, yet automated approaches to RE remain rigid. Recent multi-agent frameworks such as MARE coordinate several large language model (LLM) agents through a fixed, hand-coded action schedule, applying the same sequence of steps to every project regardless of its needs. This thesis presents REMARL, a framework that makes the orchestration strategy itself learnable. REMARL places a Proximal Policy Optimization (PPO) decision layer above a six-agent LLM pipeline that extends MARE with a sixth agent, a Negotiator, for conflict resolution. The policy observes a numeric summary of the shared workspace and selects which action an agent performs next, learning this skill from scored practice projects inside RESimEnv, a purpose-built Gymnasium-compatible simulation environment with a semantic evaluation oracle. Across twenty-two paired evaluation scenarios, the trained Collector policy outperformed a faithful re-implementation of MARE's fixed schedule with a large effect size (Cohen's $d = 1.361$, $p < 0.0001$), driven by a large gain in requirement precision. A controlled ablation showed the Negotiator significantly improves conflict resolution ($d = 1.23$, $p < 0.0001$) without harming other quality dimensions. Against a single-LLM baseline of identical model capacity, the full pipeline achieved a higher overall score and a conflict handling capability the single model structurally lacks. A human evaluation by an industry inspection team across ten scenarios favored REMARL over MARE on completeness, correctness, consistency, and traceability, and a case study on ten real-world documents from the PURE dataset showed the same overall-score and conflict-handling advantages transferring with statistical significance ($p = .0070$ and $p = .0008$ respectively). The results show that learned orchestration is a practical and measurable improvement over fixed multi-agent pipelines in requirements engineering. Code can be found at: https://github.com/sujairibrahim/REMARLThesis.git.

Keywords: *Blockchain, E-voting, Gasless transactions, Smart contracts, Privacy, Zero-Knowledge proofs.*

A Hybrid AI and Kansei Engineering Framework for Post-Release Requirement Prioritization in Banking Applications: Balancing Customer Emotions and Developer Constraints

[1]S.P.Y. Rathnasiri* and [2]P.K.D.K. Kaushalya
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing & Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*spyrathnasiri@std.appsc.sab.ac.lk

Digital banking software requirement prioritization often suffers from "frequency bias," where high-volume minor usability complaints systematically overshadow low-frequency, psychologically critical events such as security breaches and transactional failures. To address this gap, this paper introduces a novel Hybrid Artificial Intelligence (AI) and Kansei Engineering (KE) framework that integrates affective computing with transformer-based Natural Language Processing (NLP). A dataset of 715 valid multilingual user reviews harvested across five major regional mobile banking applications was processed through an automated text-preprocessing pipeline equipped with native-script normalization, translation, and emoji-to-text conversion. The core classification architecture employs a three-layer cascaded pipeline comprising a Kano-model security rule engine for immediate risk identification, a negation-aware lexicon engine mapping emotional expressions, and zero-shot BART-large-MNLI transformer inference for contextual emotional classification. A T5 transformer model subsequently synthesizes raw user feedback into 31 standardized, IEEE 830-compliant software requirement specifications grouped across core banking themes. Requirements are systematically ranked using a mathematical Hybrid Priority Score (HPS) algorithm that balances mean Kansei emotional severity against Fibonacci- scaled developer effort. Empirical evaluation demonstrates that the framework successfully elevates low-volume, high-severity security and backend stability requirements into high-priority release phases, confirming its resistance to frequency bias. Model reliability was rigorously verified through human expert validation, yielding a Cohen's Kappa coefficient of kappa = 0.64 (Substantial Agreement) and demonstrating graceful degradation across adjacent negative emotional states. Ultimately, this is the first framework to operationalize Kansei Engineering alongside transformer-based NLP for multilingual, post-release requirement prioritization in banking, offering a reliable, emotionally intelligent roadmap for the next generation of banking application development.

Keywords: *Affective computing, Natural language processing, Priority matrix, Requirements engineering, Transformer models*

Evolution-Aware LLMs for Method-Level Predictive Refactoring and Proactive Issue Prevention in Java

[1]U.Y.A.N.S.R Athukorala* and [2]S. Vasanthapriyan
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*uyansrathukorala@std.appsc.sab.ac.lk

Web-based platforms used by government educational institutions have become an important aspect of the digital system due to their growing dependence, making web security a significant concern. Public websites serve as the starting points for communication and information exchange. However, incorrectly setting up network services and a lack of protection against network attacks can make such systems vulnerable to online security risks. This paper provides a network traffic security study of selected government educational websites in Sri Lanka through the analysis of open network ports, protocol security settings, and resilience against DDoS attacks. Network scanning tools, protocol evaluation tools, and website monitoring techniques were used to collect data on open ports, services, TLS protocol setups, and website response performance. Python-based data analysis tools were used for the statistical study. Findings show that most sites have required web service ports such as HTTP and HTTPS open; however, many institutions keep management or legacy ports open, increasing the potential attack surface. While HTTPS encryption is widely used, significant differences still exist in TLS configurations. The average response time of analyzed websites was approximately 2.44 seconds, and not all institutions have implemented advanced traffic protection technologies like Web Application Firewalls (WAF) or cloud-based systems. The results highlight the need for better security measures, such as limiting unnecessary port exposure, introducing the latest TLS encryption standards, and improving system protection to resist traffic-based assaults.

Keywords: *Evolution-Aware learning, Java software maintenance, Large language models (LLMs), Predictive refactoring, Proactive software maintenance*

Vehicle Type-Based Alert System for Hazardous Pathways

[1]H.G.V.D. Dayarathne* and [2]K.G.L. Chathumini
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing & Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*hgvddayarathne@std.appsc.sab.ac.lk

Multimodal workload analysis has become extremely important for Mountain roads like the Kadugannawa pass in the Kandy District carry a very different accident risk than flat urban roads. Steep slopes, sharp blind curves, sudden weather changes, and a mix of vehicles ranging from motorcycles to heavily loaded lorries make this corridor especially dangerous, yet it is still protected only by static warning signs. This research study proposes a ML model, that predicts accident risk on this corridor while also accounting for vehicle type, since a car and a loaded lorry do not face the same risk on the same curve. Previous accident records in the Kadugannawa corridor were combined with weather data from the Open-Meteo API, road geometry and slope data extracted from OpenStreetMap, and simulated OBD-II vehicle telemetry. Over seventy features were engineered from this combined data, including interaction terms such as slope combined with vehicle type. Eleven classification algorithms were trained and compared, with F1-macro used as the primary evaluation metric because high-severity accidents form a rare, safety-critical minority class. The tuned XGBoost model performed best, achieving an F1-macro of 0.912 and an AUC-ROC of 0.980, outperforming the originally proposed Random Forest baseline. SHAP-based explain ability analysis showed that weather conditions, road curvature, and vehicle type each contribute measurably to high-risk predictions, supporting the case for vehicle- aware accident modelling. A working prototype was also built - a backend API, a web dashboard, an Android app, and a telemetry simulator - showing the model can operate as part of a real system rather than remaining an offline experiment. The study is limited to a single road corridor; future work should validate the system on other mountain roads using larger, multi-region datasets before operational deployment.

Keywords: *Accident risk prediction, Machine learning, Mountain roads, SHAP explain ability, Vehicle-type classification, XGBoost.*

Fine-Tuned BERT for Polarity Classification of IoT Product Reviews: A Cross-Corpus Evaluation

[1]A.B.M. Asloof* and [2]S. Adeeba
[1]Department of Software Engineering, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing and Information Systems, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*abmasloof@std.appsc.sab.ac.lk

Online marketplace reviews have become an important yet underused source of field-failure evidence for hardware manufacturers, particularly for Internet of Things products where users describe faults publicly long before contacting support. Reviews collected from marketplaces contain rich indicators of product defects, including outright failures, compatibility problems, power and thermal faults, and documentation gaps. However, existing approaches examine only finished consumer IoT products such as smart thermostats, cameras and voice assistants, and report classification performance solely on the corpus used for training, leaving both the component layer and the question of generalisation unaddressed. This study addresses these limitations by proposing a two-stage pipeline that first classifies review polarity and then mines fault categories from the reviews predicted negative. The study constructs a corpus of 9,893 polarity-labelled reviews spanning IoT components and consumer electronics, and derives a second corpus of 36,332 labelled reviews from a publicly released but previously unlabelled IoT review dataset. Several classical and transformer-based models were trained and evaluated under ten-fold stratified cross-validation, and every model was additionally tested under a bidirectional cross-corpus design. Among all the trained models, fine-tuned BERT revealed superior predictive performance, obtaining a macro-F1 of 94.36% with negative-class recall of 95.80%, against 93.83% for the strongest classical configuration. The advantage proved far larger under domain shift: cross-corpus macro-F1 reached 92.51% for BERT against 87.46% for the classical baseline, with a forward transfer gap of 1.64% against 7.60%, indicating that transformer benefit in this task is principally a robustness effect rather than an accuracy effect. A three-facet fault taxonomy containing four categories absent from prior consumer-IoT work was constructed and applied, with component reviews found to name a specific fault in 59.50% of cases against 31.70% for consumer electronics. Category- level findings are reported as preliminary at moderate annotation agreement of 55.00%.

Keywords: *BERT, Cross-Corpus transfer, Fault taxonomy, IoT components, Review mining*

Predictive and Prescriptive Risk Management in Agile Projects: An Explainable AI (XAI) Framework for Root Cause Prioritization

[1]A.W.A.D.D.S. Withanarachchi*, [2]L.S. Lekamge and [1]S. Sweshthika
[1]Department of Software Engineering, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing and Information Systems, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*waddswithanarachchi@std.appsc.sab.ac.lk

Agile software development projects continue to experience high rates of sprint failure, including incomplete story-point delivery, unplanned scope changes, and requirement volatility, yet organizations largely lack objective, data-driven early-warning systems to anticipate such risks. Existing predictive models, where they exist, typically operate as black boxes, offering little insight into the underlying causes of predicted risk or actionable guidance for mitigation. This research addresses this gap by developing an Explainable Artificial Intelligence (XAI) framework for predictive and prescriptive risk management in Agile software projects. The study is empirically grounded in the TAWOS dataset, from which 2,804 closed sprints across 39 open-source Jira-based projects were extracted and processed into 16 engineered features spanning Technical, Human, and Process dimensions. Sprints were labelled into three risk categories (Low, Medium, High) using percentile and threshold-based criteria applied to completion rate, reassignment count, and requirement change count. Eight supervised classifiers, Logistic Regression, Random Forest, XGBoost, Support Vector Machine, Naive Bayes, K-Nearest Neighbors, Gradient Boosting Machine, and AdaBoost, were trained. Random Forest was selected as the final predictive model, achieving the strongest overall performance, with an accuracy of 0.765, weighted F1-score of 0.765, and weighted AUC-ROC of 0.903 on the held-out test set. SHAP TreeExplainer was subsequently applied to generate global and instance-level explanations, identifying total status churn, title change count, and story point change count as the dominant drivers of sprint risk. These explanations were operationalized into a novel Risk Priority Score and mapped to ISO 31000:2018 and PMBOK-aligned prescriptive recommendations, enabling transparent, root-cause-driven, and actionable risk mitigation guidance for Agile teams and project managers, moving beyond conventional black-box prediction toward practitioner-oriented, standards-compliant decision support.

Keywords: *Agile risk management, Explainable AI, Machine learning, SHAP, Sprint-Level prediction*

Live Documentation Sync for Angular Using AST Parsing and Rule-Based Logic

[1]N.S.S. Weerasinghe* and [2]W.T.S. Somaweera
[1]Department of Software Engineering, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing and Information Systems, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*nssweerasinghe@std.appsc.sab.ac.lk

Software documentation frequently becomes outdated when source code changes are not reflected promptly in the corresponding documents. This problem is particularly significant in Angular applications because their structures include framework-specific constructs such as components, services, modules, directives, pipes, decorators, dependency injection, lifecycle hooks, routes, and Signal-based APIs. This research designed, developed, and evaluated a live documentation synchronization framework for Angular applications using Abstract Syntax Tree parsing and deterministic rule-based logic. The study followed Design Science Research Methodology and implemented the proposed framework as a Visual Studio Code extension using TypeScript, Node.js, ts-morph, and json-rules-engine. The extension tracked the Angular source files, gathered structured metadata, compared structures to previously stored snapshots, determined if documentation rules are met, and documented incrementally, adding, updating, or removing documentation as needed. The framework provides the traditional decorator based constructs of Angular, and some signal based APIs such as input(), input.required(), output(), and model(). The evaluation involved 20 Angular developers and 100 file-level technical records. The synchronized documentation achieved 100% completeness and 100% truthfulness relative to the supported extraction and documentation schema. The mean internal synchronization-processing latency was 35.25 ms, while the mean AST parsing, metadata- extraction, and rule-evaluation times were 5.36 ms, 2.24 ms, and 6.35 ms, respectively. Developer evaluation produced an overall satisfaction mean of 4.30 out of 5, indicating a favourable perception of the framework. The results show that Angular-aware AST analysis, structural change detection, rule-based decision making and IDE integration can provide a lightweight, transparent and responsive approach to maintain synchronized software documentation.

Keywords: *Abstract syntax tree, Angular, Automated software documentation, Live documentation synchronization, Rule-Based logic*

Automated Generation of Functional and Non-Functional Requirements from Agile User Stories using Large Language Models

[1]M.H.A.R.T Vishwajith* and [2]U.A.P. Ishanka
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*mhartvishwajith@std.appsc.sab.ac.lk

Agile user stories support rapid collaboration but often leave behavioural details and quality constraints implicit. This thesis proposes a large language model (LLM)-assisted Agile Requirement Clarification Framework and evaluates its generation, traceability, quality- screening, and consensus components. A source-balanced feasibility sample of 100 public user stories from 22 source files was processed by four recorded LLM–prompt configurations: Claude, GPT5, Gemini, and DeepSeek. Because the prompts differed materially, the unit of comparison was the complete configuration rather than the model alone. Three anonymised information technology (IT) industry evaluators independently assessed 5,864 generated functional and non-functional requirements (FRs and NFRs) and 359 story-level output sets. Binary judgments were consolidated by majority vote, while median ratings formed consensus scores on four 1–5 quality measures. Fleiss' kappa showed moderate agreement for validity (k = 0.540), ambiguity (k = 0.488), and duplication (k = 0.452). Mean pairwise quadratic-weighted kappa was moderate for completeness (0.521), relevance (0.500), readability (0.471), and semantic similarity (0.493). A requirement was classified as usable only when it was valid, unambiguous, and non-duplicate by consensus. Gemini achieved the highest usable rate (88.45%), GPT5 the largest usable yield (19.87 requirements per input story), and Claude the highest mean relevance (4.30) and semantic similarity (3.99). The recorded DeepSeek configuration generated requirements for 60% of stories and usable NFRs for 4%. On 59 complete cases, Friedman tests detected differences in completeness, readability, and semantic similarity, but not relevance; Kendall's W ranged from 0.043 to 0.331. The findings support the framework's evaluated core as a structured aid for early requirement clarification. Final stakeholder approval, field use, and reductions in cost, delay, or rework were not evaluated.

Keywords: *Agile requirements engineering, Functional requirements, Large language models, Non-functional requirements, User stories*

Validating LLM-Generated Software Requirements with Lightweight Formal Checks, Structured Review Support, and Evidence-Based Validation

[1]T.A.D.R.P. Chandrarathna* and [1]R. Nirubikaa
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*tadrpchandrarathna@std.appsc.sab.ac.lk

Large language models (LLMs) can draft software requirements quickly, but the generated text may contain missing fields, vague quality targets, duplicates, and conflicting numeric limits. This study developed a lightweight validation pipeline for LLM-generated requirements. The pipeline standardizes requirements into a minimal intermediate representation and applies rule-based checks for schema quality, ambiguity, vacuous metrics, near-duplicates, and numeric conflicts. It also produces simple explanations and minimal conflict cores for review. The evaluation used 353 generated requirements from four system groups: LMS, SMS, MBD, and RSV. The standardization stage repaired 29 structurally shifted records and improved key schema-conformance measures to 100%, while required-field completeness remained at 99.43%. The controlled labelled benchmark contained 15 injected defects, including five numeric-conflict pairs and five vacuous metrics. The checker detected all injected defects, but this result is reported only as a controlled benchmark outcome. To avoid overclaiming, an end-to-end balanced evaluation score was also calculated using RQ1 defect-detection F1, RQ2 reviewer-proxy F1, RQ3 evidence-alignment accuracy, and RQ3 violation-detection F1. The balanced score was 93.95%. A simulated reviewer proxy improved defect recall from 0.333 to 1.000 and reduced detailed-review scope by 60.3%. A controlled synthetic operational-evidence experiment improved alignment accuracy from 0.500 to 0.841 and reduced incorrectly approved violations from 12 to 0. The results show technical feasibility, while future work should validate the framework with real reviewers and real application evidence.

Keywords: *Formal checks, Human-in-the-Loop, Large language models, Requirement validation, Requirements engineering*

Lightweight AI Models for Mobile Devices: A Comparison of Optimization Techniques for Performance and Energy Efficiency

[1]G.H.T. Wijerathna*, [1]W.D.D. Lakshan and [2]A.I. Hewarathne
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]AI and Technology, Boolean Labs, Sri Lanka.
*ghtwijerathna@std.appsc.sab.ac.lk

Artificial intelligence (AI) is now widely used in mobile applications, but running AI models directly on smartphones is still difficult because mobile devices have limited memory, processing power, and battery capacity. Even lightweight models such as MobileNetV2 and ShuffleNetV2 can increase inference time and battery consumption. To overcome these challenges, researchers have introduced model optimization techniques such as pruning, quantization, and knowledge distillation. However, previous studies evaluate these techniques on desktop computers or simulation environments rather than real-world mobile devices such as smartphones and tablets, making it difficult to understand their practical performance. This study compares the performance of the original MobileNetV2 model with four optimized versions: a pruned architecture, a dynamically quantized architecture, an INT8 quantized architecture, and a knowledge distilled architecture. All models were trained using the CIFAR- 10 dataset. The baseline model achieved an accuracy of 86.33%. After conversion to TensorFlow Lite, each model was tested on a real Android smartphone using a custom built benchmarking application. Every architecture was evaluated over 30 independent runs on a real Android mobile device, measuring inference latency, RAM usage, estimated energy consumption, and battery usage. The results show that each optimization technique has its own strengths. The knowledge distilled model delivered the fastest inference time (0.58 ms) and the lowest energy consumption while maintaining 84.34% accuracy. The pruned model achieved the highest accuracy (85.95%) and improved inference speed by about 43%, offering a good balance between performance and accuracy. Although the INT8 model ran much faster, its accuracy dropped by 19.65%, making it less suitable for applications where prediction accuracy is important. Overall, the experimental results demonstrate that knowledge distillation is the most suitable optimization technique for resource-constrained mobile devices when inference speed and energy efficiency are prioritized, whereas pruning provides the best balance between prediction accuracy and computational efficiency.

Keywords: *Artificial intelligence, Knowledge distillation, Model optimization, Pruning, Quantization*

DevSecOps Integration with AI-Driven Security Automation: An Empirical Evaluation of Traditional and AI-Enhanced Free/Open-Source Security Scanning Tools

[1]L.H.S. Fernando* and [2]S. Vasanthapriyan
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*lhsfernando@std.appsc.sab.ac.lk

The integration of AI-driven security automation into DevSecOps CI/CD pipelines presents a critical strategic decision for software engineering organisations. Yet, practitioners face limited empirical evidence comparing traditional and AI-driven free/open-source security tools under equivalent conditions. This study addresses this gap through a controlled empirical evaluation of six free/open-source security scanning tools: three traditional (SonarQube Community Edition, Bandit, Trivy), two AI-enhanced (Semgrep, GitHub CodeQL), and one dynamic analysis tool (OWASP ZAP), evaluated against two target applications: a purpose-built vulnerable Python Flask API and OWASP WebGoat (v2023.8). An integrated DevSecOps security pipeline was implemented using GitHub Actions CI/CD infrastructure, combining Bandit, Semgrep, Trivy, and CodeQL within a single automated workflow to demonstrate real-world AI-driven security automation. Each tool was executed three times for each application, and findings were assessed against a predefined ground truth of 12 vulnerabilities per application using precision, recall, and F1-score. Results show that Bandit (traditional) achieved the highest F1-score on the Python application (0.71), while CodeQL (AI-enhanced) achieved the highest F1-score on the Java application (0.67), challenging the assumption that AI-enhanced tools consistently outperform traditional alternatives. Tool specialisation and language-specific design emerged as stronger predictors of detection performance than the traditional/AI categorisation. Trivy and OWASP ZAP each identified vulnerability categories invisible to code-level scanning tools, demonstrating strong complementarity across tool categories, while no tool detected access-control-related vulnerabilities, indicating a persistent automated-detection blind spot. All six tools evaluated demonstrated perfect precision, indicating no false positives within the scope of this study. A practitioner decision framework is proposed to guide DevSecOps tool selection according to organisational context and CI/CD pipeline requirements.

Keywords: *AI-driven security automation, CI/CD pipeline security, DAST, DevSecOps, empirical evaluation, open-source security tools, SAST, vulnerability detection*

An Empirical Survey on the Impact of State-Management Solutions on React Developer Satisfaction and Perceived Project Scalability

[1]D.M.N.D. Dissanayakaa* and [2]K.G.L. Chathumini
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*dmnddissanayaka@std.appsc.sab.ac.lk

State management influences how React applications are understood, modified, debugged, and extended, yet tool-selection decisions are frequently based on popularity or personal preference rather than integrated empirical evidence. This study examines the impact of React state-management solutions on developer satisfaction and perceived project scalability. A cross-sectional quantitative survey was analysed using a main de-duplicated sample of 200 eligible React users, with the full set of 288 eligible submissions retained for sensitivity analysis. The instrument measured satisfaction, cognitive and debugging challenges, and perceived scalability using five-point Likert items. Internal consistency, sampling adequacy, exploratory factor analysis, descriptive statistics, Kruskal–Wallis tests, Holm-adjusted post-hoc comparisons, Spearman and Pearson correlations, robust multiple regression, and a survey-only multi-criteria decision analysis were applied. Satisfaction and perceived scalability showed strong reliability (Cronbach's alpha = .931 and .876 respectively), while the challenge scale showed moderate reliability (alpha = .654). Satisfaction differed significantly across primary tools ($H = 21.509$, $p = .0015$), whereas perceived scalability did not differ significantly ($H = 10.139$, $p = .1189$). Productivity, ease of learning, documentation, and collaboration were positively associated with satisfaction. Developer satisfaction was strongly related to perceived scalability (Spearman rho = .682, $p < .001$), and remained a significant predictor after controlling for challenges, experience, team size, and project complexity. Context API, Zustand, and Redux Toolkit ranked highest in the survey-only decision model among groups with at least ten respondents. Because the questionnaire measured perceptions rather than objective runtime or codebase metrics, conclusions concern perceived scalability. The compressed timestamp window and identical response patterns also require cautious interpretation. The study contributes an integrated developer-centred framework and practical selection guidance while identifying priorities for benchmark-based and longitudinal validation.

Keywords: *Developer experience, Developer satisfaction, Perceived project scalability, React, State management*

Expertise Mapping from Developer Communications: A Transformer-Based Pipeline for Knowledge Graph Construction and Explainable Expert Recommendation

[1]M.T.S. Charuka* and [2]G.A.C.A. Herath
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*mtscharuka@std.appsc.sab.ac.lk

Locating the right expert in a distributed software team is a long-standing problem, and the evidence needed to solve it already sits in developer conversations. This thesis presents a pipeline that turns that evidence into a queryable expertise knowledge graph supporting explainable recommendation. A corpus of 3,919 comments from a large open-source repository was annotated under documented guidelines at three levels, communicative intent, technical entities, and semantic relations, yielding 12,898 entity and 5,239 relation annotations. Three CodeBERT models were fine-tuned on these tasks. Comment classification reached 0.816 macro F1 and relation extraction 0.829, both inside the target band of 0.80 to 0.90 fixed before evaluation, while entity recognition reached 0.735 exact-span F1, below the band. Model outputs populate a graph of 414 pseudonymized developers and 2,865 deduplicated concepts, over which a query component ranks developers by a transparent label-weighted rule and returns verbatim evidence with every recommendation. Under an identical protocol the pipeline outperforms a keyword-frequency baseline by 0.283 micro F1 on entity extraction, concentrated in the types where surface form does not determine the label. A controlled experiment then held the pipeline constant while the annotation guidelines were tightened. Correcting inconsistent labels lowered the measured precision of the affected class from 0.42 to 0.38, because the model had learned the same ambiguity present in the annotations, so its errors and the annotation errors coincided and scored as correct. Label noise can therefore inflate reported performance as well as depress it. In a blinded comparison, practitioners preferred the component to the keyword baseline in 22 of 28 judgements. The work contributes a reusable annotated corpus with its guidelines, evidence on how annotation consistency governs both model performance and the validity of its measurement, and a privacy-preserving architecture from raw comment text to recommendations a user can contest.

Keywords: *Annotation quality, Expertise mapping, Explainable recommendation, Knowledge graphs, Named entity recognition*

Explainable Cross-Project Bug Severity Prediction using Lightweight Machine Learning

[1]L.M.P.N. Bandara* and [2]U.A.P. Ishanka
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*klmpnbandara@std.appsc.sab.ac.lk

Automated bug severity prediction helps software maintenance teams prioritize defects, but existing approaches face a persistent trade-off: high-performing models are often computationally heavy, opaque, and evaluated only within a single project, whereas lightweight, interpretable models are rarely tested across projects or explained. This study addresses that gap by developing and evaluating an explainable, lightweight machine learning pipeline for cross-project bug severity prediction, using three Bugzilla-based datasets (Mozilla, Eclipse, freedesktop.org) comprising 27,457 bug reports unified under a three-level severity scheme (Low, Medium, High). Bug-report text was represented using TF-IDF (a choice empirically validated against Word2Vec and FastText embeddings), with stop word removal, combined with eight handcrafted metadata features, and five lightweight algorithms, namely Logistic Regression, Linear SVM, Decision Tree, Random Forest, and XGBoost. Each model was trained and evaluated under a leave-one-project-out cross-project design. Class imbalance, with medium severity comprising roughly 77 percent of reports, was addressed through a controlled majority-class reduction technique rather than conventional resampling. XGBoost achieved the best performance (macro-F1 = 0.415, MCC = 0.172), narrowly ahead of Random Forest, and a supplementary within-project validation showed that the cross- project generalization gap was small, at most 3.53 macro-F1 points, indicating that cross-project transfer is not the primary limiting factor on performance. SHAP, LIME, and native feature importance analyses confirmed that the model's predictions were grounded in domain-plausible crash, error, and severity-register vocabulary rather than noise, and further revealed that several misclassifications involved severity labels not well supported by the report's own text. These findings, interpreted alongside prior evidence of substantial severity-label inconsistency, indicate that the observed performance ceiling reflects the inherent subjectivity of human-assigned severity labels rather than a deficiency in the modelling approach, supporting the pipeline's value as an explainable decision-support tool for bug triage prioritization.

Keywords: *Bug severity prediction, Class imbalance, Cross-Project defect prediction, Explainable AI (XAI), Lightweight machine learning*

A Hybrid NLP-Based Framework for Ambiguity Detection in Software Requirements

[1]R. Sangeerthana* and [1]S. Sweshthika
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*rsangeerthana@std.appsc.sab.ac.lk

Ambiguity in natural-language software requirements can lead to conflicting interpretations, implementation errors, rework, and increased development effort. Existing automated ambiguity-detection approaches support Requirements Engineering; however, they may struggle to capture contextual meaning and often operate as one-stage detectors without verifying whether analyst revisions have successfully resolved the identified ambiguity. This study aimed to compare rule-based, classical machine-learning, deep-learning, and transformer-based approaches for detecting lexical, modal, and referential ambiguity, and to develop a human-in-the-loop framework that re-evaluates revised requirements for unresolved or newly introduced ambiguity. A total of 5,000 requirement statements were extracted from publicly available SRS documents. Three independent software-engineering annotators manually assigned separate binary labels for lexical, modal, and referential ambiguity, thereby constructing a multi-label dataset. Majority voting was used to establish the final reference labels. Rule-based methods were evaluated first, followed by traditional machine-learning classifiers using TF-IDF, Word2Vec, and FastText feature representations, deep-learning approaches including BiLSTM and CNN-BiLSTM, and transformer-based models including BERT and ELECTRA. All approaches were comparatively evaluated using accuracy, ROC-AUC, and Matthews Correlation Coefficient. The best-performing model was subsequently integrated into a prototype supporting automated ambiguity detection, human revision, conflict identification, and iterative re-evaluation. Among the evaluated approaches, BERT achieved the best overall performance with an accuracy of 0.9960, a ROC-AUC of 0.9983, and an MCC of 0.9862. The integrated prototype supported iterative human revision and automated re-evaluation, enabling the identification of ambiguity that remained unresolved or was newly introduced during requirement correction. The findings demonstrate that combining contextual NLP-based ambiguity detection with iterative human review can prevent weak revisions from being accepted without verification. The proposed framework extends conventional one-stage ambiguity detection into a structured and traceable requirements-refinement process.

Keywords: *Ambiguity detection, BERT, Natural language processing, Software requirements*

A Machine Learning Approach for Recommending Suitable Technology Stacks Based on Project Characteristics

[1]S. Kanujan* and [2]H.M.K.T. Gunawardhana
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*kkanujan@std.appsc.sab.ac.lk

Selecting an appropriate technology stack is one of the most consequential decisions in software development, directly influencing a project's performance, scalability, security, and long-term maintainability. Despite its significance, this decision continues to rely heavily on developer experience and personal preference rather than evidence-based analysis, often resulting in poor architectural choices, increased technical debt, and project failure. This research proposes and evaluates a machine learning-based recommendation system that predicts suitable frontend frameworks, backend technologies, and database systems from project characteristics. A dataset of 4,850 real-world software projects was compiled from GitHub repositories and collaborating software companies across ten application domains, including e-commerce, healthcare, education, finance, social media, logistics, real estate, human resources and recruitment, IoT and smart systems, and travel and hospitality. Following systematic data cleaning, 4,437 valid records were retained. Exploratory analysis showed that project domain and primary programming language are the strongest predictors of technology stack selection, with language-to-backend mappings reaching 100% consistency in five cases. Feature engineering produced a 394-dimensional input vector combining ordinal encoding, one-hot encoding, and TF-IDF vectorization of functional and non-functional requirement text. Five machine learning approaches were evaluated: classifier-chain XGBoost, classifier-chain CatBoost, LightGBM, a NoBERT model (combining frozen Sentence Transformer embeddings with a LightGBM classifier), and a stacking ensemble integrating XGBoost, CatBoost, LightGBM, and NoBERT. The classifier-chain XGBoost model achieved the strongest performance, with frontend accuracy of 76.6%, backend accuracy of 71.6%, and database accuracy of 80.0%, an average of 76.1%, substantially exceeding a majority-class baseline. A functional prototype web application, developed using Flask, allows practitioners to enter project requirements and receive ranked technology recommendations together with confidence scores and feature-based explanations. The findings demonstrate that data-driven methods can meaningfully support technology stack decisions, reducing subjectivity and improving consistency in early-stage software planning.

Keywords: *Feature engineering, Machine learning, Recommendation system, Software engineering, Technology stack selection*

ML-Based Performance Bug Prediction in Serverless and Cloud-Native Applications

[1]J. Shanchikka*, [2]W.V.S.K. Wasalthilaka and [3]K. Srisaiyeegharan
[1]Department of Software Engineering, Sabaragamuwa University of Sri Lanka
[2]Faculty of Computer Science, Technical University of Dresden, Germany
[3]Telstra Group Limited, Melbourne, Victoria, Australia
*jshanchikka@std.appsc.sab.ac.lk

Understanding and predicting performance bugs in serverless applications is important for maintaining reliability, controlling operational costs, and supporting consistent user experiences. Existing studies commonly examine static code metrics, workload behaviour, or runtime telemetry separately and frequently focus on detecting anomalies after production deployment. This study addresses this gap by developing a multi-dimensional machine-learning framework that combines static code metrics, workload characteristics, and AWS Lambda telemetry to identify performance-bug risk during controlled pre-production staging. A reproducible dataset containing 6,268 unique invocations from 18 serverless functions was prepared, comprising 4,878 normal executions and 1,390 performance-bug cases. The dataset was labelled through a human-guided, rule-based procedure combining known fault-injection context, function-relative baseline thresholds, and measured performance impact. Six tree- based classifiers Random Forest, XGBoost, LightGBM, CatBoost, HistGradientBoosting, and ExtraTrees were evaluated using four feature-availability configurations and a stratified 70/15/15 train, validation, and test split. LightGBM achieved the highest realistic Tier B test F1-score of 0.8256, with precision of 0.8485 and ROC-AUC of 0.9669. XGBoost achieved a closely comparable F1-score of 0.8244, precision of 0.8408, recall of 0.8086, and ROC-AUC of 0.9660. LightGBM ablation analysis showed that the complete code, workload, and cloud feature set achieved the highest F1-score of 0.8256 among the seven feature-group combinations, while cloud telemetry was the strongest individual dimension with an F1-score of 0.7867. SHAP analysis of the saved LightGBM Tier B champion model generated feature- level explanations for a reproducible sample of 500 test records. Error analysis of the LightGBM champion model identified 41 false negatives and 30 false positives on the 941- record test set. The findings demonstrate that multi-dimensional staging telemetry can support effective pre-production performance-risk assessment for known serverless functions, although row-level splitting and proximity between some telemetry features and the labelling rules limit broader generalisation.

Keywords: *AWS Lambda, Cloud telemetry, Machine learning, Performance bug prediction, Serverless computing*

LLM-Powered Automated Root Cause Analysis for Microservices using Single-Modality Log Analysis

[1]A.I.G.A.V. Alakolanga* and [2]S. Vasanthapriyan
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*aigavalakolanga@std.appsc.sab.ac.lk

Modern microservices architectures comprising tens to hundreds of independently deployable services have introduced significant challenges in failure diagnosis, with manual diagnosis reported in the literature to commonly require substantial engineer time per incident, and existing statistical AIOps tools reported to exhibit high misclassification rates in cascading failures. This study developed a GPT-4o-powered automated root cause analysis (RCA) pipeline that operates exclusively on log data, combining Drain3- based log template extraction, structured per-service log representation, and three alternative GPT-4o prompting strategies of increasing structure, evaluated against a transparent, non-LLM statistical baseline. The evaluation used twenty controlled failure-injection scenarios on the Sock-Shop microservices testbed, comprising ten distinct failure types each independently repeated once to assess prediction consistency across trials. The proposed system's primary configuration achieved 75.0% root-cause identification accuracy overall, and 80.0% on the subset of scenarios common to all three prompting strategies, compared to 20.0% and 40.0% respectively for the statistical baseline, with an average analysis latency of 4.61 seconds and 90.0% prediction consistency across repeated trials. All three prompting strategies achieved identical accuracy on the scenarios where they were directly compared, indicating that log evidence availability, rather than prompt structure, was the dominant factor influencing accuracy. The system consistently and reproducibly failed, through both trials, on two failure types in which the root-cause service produced no log output during failure. To avoid overclaiming, this failure boundary is reported alongside the accuracy results rather than treated as a solved case. The results show technical feasibility, while future work should validate the approach on additional testbeds and with independent expert-rated report review.

Keywords: *AIOps, GPT-4o, Large language models, Microservices, Root cause analysis*

Software Architecture Evaluation and Refactoring for Python Systems using Machine Learning-Based Code Smell Detection

[1]D. Dilukshan*, [1]W.M.L.S. Abeythunga and [1]S. Nishankar
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*djdilukshan@std.appsc.sab.ac.lk

Code smells are indicators of poor design or implementation choices that degrade software maintainability over time. Despite Python's widespread use in modern software and data science systems, comparatively little research has addressed machine learning-based code smell detection for the language. This study presents a machine learning-based framework for detecting Python code smells and recommending corresponding refactoring actions. A labeled dataset of over 150,000 instances (CodeSmellData, Velasco et al., 2024) was analysed at the function level using Pylint static analysis, from which thirteen empirically-derived smell categories were identified. Analysis revealed that 55% of instances shared identical feature vectors with conflicting labels, an irreducible ceiling on classification accuracy; deduplication reduced the usable dataset to 18,402 clean instances. Five classifiers — CatBoost, XGBoost, LightGBM, Random Forest, and SVM — were evaluated under equal tuning budgets, with LightGBM selected after significantly outperforming CatBoost and XGBoost (Wilcoxon signed-rank test, $p < 0.05$ in both cases). Iterative, error-driven feature engineering — in which qualitative misclassification analysis directly informed new Abstract-Syntax-Tree-derived structural features — substantially improved performance. Subsequent diagnosis of a 10.3-point train/test accuracy gap led to the application of regularization, formally confirmed ($p = 0.0020$) to significantly reduce overfitting across cross-validation folds. The final model achieved 88.63% test accuracy and a macro F1-score of 0.8567 across thirteen categories, a result confirmed stable across cross-validation, validation, and held-out test partitions. Beyond detection, a deterministic, rule-based refactoring recommendation layer maps each detected smell to an established refactoring strategy, demonstrated through a working prototype and qualitative case studies on real Python functions. The study further identifies, as an explicit limitation, that four of the evaluated smell categories were flagged by the source dataset's own creators as exhibiting low label precision, a caveat disclosed to guide responsible interpretation of the results.

Keywords: *Code smell detection, Machine learning, Python, Refactoring recommendation, Software maintainability*

Evaluating and Redesigning the Usability of Sri Lankan Government Websites for Improved User Experience and Accessibility - A Case Study Approach

[1]M. Akshayan* and [2]H.M.K.T. Gunawardhana
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*makshayan@std.appsc.sab.ac.lk

Government websites play a vital role in providing citizens with access to public information and essential digital services. The effectiveness of these services depends not only on the availability of information but also on the usability, accessibility, and overall user experience of the websites. This research evaluates and improves the usability of three selected Sri Lankan government websites: the University Grants Commission (UGC), Sri Lanka Railways, and the Department of Immigration and Emigration through an evidence-based UX/UI redesign process. The study began by evaluating the existing websites to identify representative user workflows and usability issues related to navigation, information organisation, accessibility, interface consistency, and workflow efficiency. Based on the identified usability problems, redesigned interfaces were developed using user-centred UX/UI design principles to improve critical workflows while preserving the original functionality of each website. Comparative usability testing was then conducted by asking participants to complete the same representative tasks using both the existing websites and the redesigned interfaces. During the evaluation, Task Completion Time was recorded to measure workflow efficiency, while the System Usability Scale (SUS) and the User Experience Questionnaire (UEQ) were used to assess perceived usability and overall user experience. The collected data were analysed to compare the usability performance of the existing and redesigned websites. The findings indicate that the redesigned interfaces enabled users to complete tasks more efficiently while achieving improved usability and user experience compared with the existing websites. The study demonstrates that an evidence-based UX/UI redesign process can effectively enhance the usability and accessibility of Sri Lankan government websites. Furthermore, the research provides practical recommendations for improving navigation, information organisation, accessibility, and workflow efficiency in government digital services and presents a structured methodology that may serve as a reference for future usability evaluation and UX/UI redesign studies involving public sector websites.

Keywords: *Accessibility, Government websites, Usability evaluation, User experience, UX/UI redesign.*

HybridBugNet: An Explainable Hybrid CodeBERT–Graph Attention Network Architecture for Method-Level Java Bug Detection

[1]M.S.M. Insari* and [2]U.A.P. Ishanka
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*msminsari@std.appsc.sab.ac.lk

Software defects remain a persistent and costly challenge in modern software engineering, with increasing system complexity rendering manual code review increasingly impractical. Existing automated bug detection approaches are constrained by their reliance on single-modality code representations and by the absence of implemented explainability mechanisms necessary for developer trust. This study proposes HybridBugNet, a hybrid neural architecture integrating CodeBERT with a three-layer Graph Attention Network through a dynamic attention fusion gate, incorporating a novel diff context mechanism to address the near-syntactic identity challenge inherent in method-level bug-fix pair classification. A balanced dataset comprising 3,706 labelled Java method samples was constructed by mining 2,721 bug-fixing commits from 14 open-source repositories spanning six software domains. Per-repository stratified splitting preserved class balance while preventing repository-level data leakage. HybridBugNet was evaluated against traditional machine learning baselines, single-modality neural models, and multiple fusion variants through comprehensive ablation experiments. The proposed model achieved an F1-score of 0.742, an AUC of 0.855, and an accuracy of 73.6% on an independent test set. Ablation analysis demonstrated that incorporating diff context was essential for reliable method-level bug discrimination, while attention gate analysis confirmed effective integration of semantic and structural code representations. Confidence calibration further supported the interpretability of model predictions. Furthermore, a composite loss function was introduced to mitigate fusion gate collapse, enabling balanced utilization of semantic and structural representations. These findings demonstrate that integrating semantic and structural code representations through an explainable dynamic fusion mechanism significantly improves automated Java bug detection while providing interpretable predictions that support developer decision-making.

Keywords: *Bug detection, CodeBERT, Explainability, Graph attention network, Hybrid architecture*

Deep Learning-Based Privacy Threat Classification Framework for Bluetooth Low Energy Communication Networks

[1]B.W.N.G.P. Shamika*, [1]Y.M.S. Tharaka and [1]W.V.C. Maduwanthi
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*bwngpshamika@std.appsc.sab.ac.lk

Bluetooth Low Energy (BLE) has become the dominant connectivity standard for wearables, fitness trackers, and other personal IoT devices, yet its built-in privacy protections, most notably MAC address randomization, have repeatedly been shown to fail against tracking and identifier-leakage attacks. Because privacy-risky BLE behavior occurs through legitimate protocol operation, it is indistinguishable from normal traffic to conventional security tools, and existing detection approaches rely almost entirely on slow, manual, expert-driven traffic analysis. This research addresses that gap by developing an automated deep learning framework for classifying BLE privacy threats. A seven-category privacy threat taxonomy, comprising Normal traffic and six threat types, tracking, identifier leakage, unauthorized data transmission, behavioral profiling, location privacy, and metadata leakage, was constructed from three public data sources and labelled using vulnerability signatures drawn from the literature. Three deep learning architectures, a Convolutional Neural Network, a Long Short-Term Memory network, and a hybrid CNN-LSTM model, were designed, trained, and benchmarked against three conventional machine learning classifiers, Random Forest, XGBoost, and Support Vector Machine, on a combined dataset of 34,793 labelled samples. XGBoost achieved the strongest overall performance, with 75.15% test accuracy, an F1-score of 75.12%, and an AUC of 0.954, while also being the fastest and among the smallest models evaluated. Five-fold cross-validation and McNemar's statistical testing confirmed the stability and significance of these results. SHAP-based explainability analysis identified signal strength and temporal features as the strongest overall predictors, while packet-level features were most important for wearable-device-specific threats. Three of the seven threat categories were classified with near-perfect accuracy, while four behaviorally similar categories showed measurable confusion, reflecting genuine overlap in their underlying signal and temporal patterns. This research contributes a reusable labelled dataset, a validated benchmark of deep learning versus conventional classifiers for BLE privacy detection, and an explainability-driven feature analysis that together provide a foundation for future real-time BLE privacy monitoring tools.

Keywords: *Bluetooth low energy, CNN-LSTM, Deep learning, Explainable AI, Privacy threat classification.*

Quantifying AI-Induced Technical Debt: An Analytical Framework for the Maintainability of LLM-Generated Java Code

[1]K.M.P.K. Bandara* and [2]B.T.G.S. Kumara
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*kmpkbandara@std.appsc.sab.ac.lk

The increasing use of large language model-based coding assistants has raised concerns about the maintainability of AI-generated source code. Although tools such as ChatGPT, GitHub Copilot, and Gemini can generate Java code rapidly, their outputs may introduce complexity, code smells, duplication, reliability issues, security concerns, and coding-standard violations. This research developed an analytical framework to quantify AI-induced technical debt in LLM-generated Java code. A balanced dataset of 200 Java files was created using 50 algorithms from ten categories. Each algorithm was represented by Human, ChatGPT, Copilot, and Gemini implementations. SonarQube, PMD, and Checkstyle were used to extract static- analysis metrics. The metrics were normalized and organized into five dimensions: Complexity, Code Smells, Duplication, Security/Reliability, and Style Violations. These dimensions were combined using a weighted equation to calculate an AI Technical Debt Score from 0 to 100 and classify each file as LowDebt, MediumDebt, or HighDebt. The results showed that ChatGPT achieved the lowest average technical debt score of 6.84, followed by Copilot with 8.54, Human with 13.84, and Gemini with 13.86. Among the 200 files, 188 were classified as LowDebt and 12 as MediumDebt. No HighDebt files were identified. Complexity was the most common debt contributor. A supporting regression model achieved an MAE of 0.0690, RMSE of 0.6667, and R2 of 0.9953. A K-Means binary experiment achieved a balanced accuracy of 0.9707. The proposed framework provides a systematic and interpretable method for assessing maintainability-related technical debt in AI-generated Java code.

Keywords: *AI-Generated code, Java, Large language models, Maintainability, Technical debt*

Reflection-Enhanced Multi-Agent RAG with Dynamic Memory for Automated Root-Cause Analysis in Software Incidents

[1]D.G.W.T. Rathnayake* and [2]G.A.C.A. Herath
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*dgwtrathnayake@std.appsc.sab.ac.lk

Diagnosing the root cause of software incidents from system logs is a time-consuming and expertise-dependent task. Although Large Language Models (LLMs) can generate plausible explanations, they are prone to hallucination when used without reliable supporting evidence. Retrieval-Augmented Generation (RAG) addresses this limitation by grounding responses in retrieved knowledge; however, most existing RAG-based root-cause analysis systems rely on static knowledge bases that do not adapt based on previous diagnostic experience. This study proposes, implements, and evaluates a reflection-enhanced multi-agent RAG system with dynamic memory for automated root-cause analysis. The proposed architecture consists of two stages. A fine-tuned ModernBERT classifier first identifies the severity of log chunks and forwards only significant incidents to a four-agent pipeline responsible for retrieval, reasoning, reflection, and memory updating. The reflection mechanism evaluates the usefulness of retrieved incidents and updates their relevance scores in a ChromaDB knowledge base, allowing future retrieval to adapt over time without retraining the language model. The system was evaluated using benchmark datasets, ablation studies, retrieval metrics, reference- free RAG evaluation, and memory evolution experiments. The classifier achieved 94.8% accuracy on the held-out test dataset and reduced expensive language model calls by approximately 20 times under a simulated 5% incident rate. The results showed that reflection-driven adaptive memory became stable after introducing deterministic filtering, bounded score updates, ensemble reflection, and exploration mechanisms while maintaining retrieval performance comparable to static memory. The proposed multi-agent architecture also provided an auditable reasoning process and produced higher-quality diagnostic reports than the baseline approaches. Overall, the study demonstrates that reflection-driven adaptive memory can be safely integrated into a retrieval-augmented multi-agent system when supported by appropriate control mechanisms, enabling continuous memory adaptation without retraining the underlying language model.

Keywords: *Dynamic memory, Large language models, Multi-Agent systems, Retrieval-Augmented generation, Root-Cause analysis*

AI-Powered Multi-Platform Job Aggregation and Recommendation System for Sri Lanka

[1]D.P.P.H.N. Thotillagolla* and [2]S. Vasanthapriyan
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*dpphnthotillagolla@std.appsc.sab.ac.lk

Job searching in Sri Lanka is still scattered across many online portals. Seekers often move between sites such as TopJobs and other local platforms, compare vacancies by hand, and struggle to find roles that fit their skills, experience, and career goals. Global job sites and general AI chatbots can offer broad advice, but they usually do not bring Sri Lankan vacancies together in one place or rank them carefully against a user's CV. This research presents an AI-powered all-in-one career guidance and job listing chatbot platform for Sri Lankan job seekers. The system aggregates job opportunities from multiple local sources, analyses user CV data, and provides personalized multi-domain job recommendations and career guidance through a conversational interface. A recommendation engine was developed using CV parsing, hybrid candidate generation with Sentence-BERT and BM25, graded relevance labelling, and a stacking ensemble of Support Vector Machine, Logistic Regression, and XGBoost. Model performance was assessed mainly with NDCG@10 under GroupKFold validation grouped by CV identity to reduce data leakage. The tuned stacking ensemble achieved a mean NDCG@10 of 0.9661 and a mean Precision@10 of 0.9415 on the training evaluation setting, outperforming the individual baseline models. Feature analysis showed that field match and skill overlap were among the strongest contributors to ranking quality. Overall, the study shows that combining local job aggregation with CV-aware recommendation and chatbot-based guidance can reduce search fragmentation and improve the relevance of job suggestions for Sri Lankan seekers.

Keywords: *CV-Based job recommendation, Job aggregation, Machine learning, Natural language processing (NLP), Semantic similarity*

Improving Energy Efficiency in Legacy Systems through Refactoring

[1]K.F. Haseefa* and [1]H.M.C. Nirmani
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*haseefsfathima2001@gmail.com

Modern software systems operate continuously and at large scale, making software inefficiency an increasing contributor to energy consumption and carbon emissions. Legacy systems are particularly vulnerable because prolonged maintenance can introduce code smells such as God Class, Long Method, Feature Envy, Shotgun Surgery, and Duplicated Code, which increase code complexity and reduce maintainability. This study investigates whether a data-driven approach can identify energy-related hotspots in legacy software and prioritize refactoring actions to reduce energy-related risks. A curated dataset of 120,000 source-file records covering six programming languages, fifteen code smell categories, and eleven refactoring techniques was used. Since direct hardware-based energy measurement was beyond the scope of the study, software quality metrics, including cyclomatic complexity, lines of code, technical debt, maintainability index, and bug- prone score, were used as indicators of energy-related risk. A composite hotspot score was developed, and a Random Forest classifier was trained to identify high-risk source files. Statistical analyses were also conducted to evaluate the effectiveness of different refactoring techniques. The Random Forest classifier achieved 93.65% test accuracy, which improved to 93.84% after hyperparameter tuning, with an AUC of 0.945. Cyclomatic complexity, technical debt, and bug- prone score were identified as the most influential predictors of hotspot status. Refactoring reduced average cyclomatic complexity by 37.2% ($p < 0.001$), with Rename Variable and Introduce Parameter Object demonstrating the greatest improvements. Based on these findings, a six-stage CI/CD-integrated framework for energy-aware refactoring is proposed. The results demonstrate that combining software quality metrics with machine learning can effectively identify energy-risk hotspots and support targeted refactoring in legacy systems. However, future research should validate the proposed framework using direct hardware-based energy measurements.

Keywords: *Code smells, Energy-Aware refactoring, Legacy software systems, Sustainable software engineering*

Evaluating the Effectiveness of Automated Web Testing Tools using Software Quality Metrics

[1]M.F.M. Fashan* and [1]B.R. Madurangi
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*mfmfashan@std.appsc.sab.ac.lk

Automated testing has become extremely important for maintaining software quality in fast-release web development, especially in JavaScript-based projects where teams must choose among frameworks with very different architectural philosophies. Existing comparative studies do not analyze framework performance across multiple independent systems and multiple quality dimensions simultaneously, because they solely rely on single-metric benchmarks or evaluate a single codebase in isolation for framework selection. This study addresses this limitation by proposing a controlled, multi-system, multi-metric empirical framework that combines defect- detection accuracy, code coverage, execution efficiency, and test-suite maintainability to compare three widely adopted testing frameworks: Jest, Cypress, and Playwright. The study extracts performance data from 108 controlled observations collected across four independently developed React/Django web applications an Inventory Management System, a Student Management System, a Hospital Management System, and a Vehicle Management System each subjected to a standardized five-defect injection protocol. Non-parametric statistical hypothesis testing was applied to evaluate framework differences across five quality metrics. Among all three frameworks, Jest and Playwright revealed statistically equivalent superior defect-detection performance, achieving mean Defect Detection Rates of 78.1% and 80.2% respectively, both significantly outperforming Cypress at 45.6% (Kruskal-Wallis $H(2) = 22.10$, $p < 0.001$). Moreover, Playwright achieved the highest mean code coverage (96.6%) and maintainability score (41.2), while Jest recorded the fastest mean execution time (6.8 seconds). The study also identified two previously undocumented behavioral anomalies a Cypress assertion-depth failure cascade and a Playwright baseline-masking effect and presents a weighted decision-scoring framework, validated through sensitivity analysis, that ranks Playwright as the optimal general-purpose framework while identifying Jest as the preferred choice when execution speed is prioritized.

Keywords: *Automated testing, Cypress, Defect detection rate, Jest, Playwright, Software quality metrics*

A Digital Twin of an Organization (DTO)-Based Predictive Business Impact Simulation using Rule Based Method

[1]R.A.D.Kumari*, [2]H.M.K.T. Gunawardhana and [2]A.S.A.Perera
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*radkumari@std.appsc.sab.ac.lk

Software engineering organizations are dynamic environments where interconnected human, process, and technical assets interact in complex ways, yet conventional project management approaches fail to anticipate cascading business impacts before they materialize. A structured survey of 301 software industry professionals confirmed this gap: 87% expressed a clear need for predictive business impact tools, 74.4% agreed that project delays cause significant damage to client trust and business continuity, 41.8% reported unexpected mid-project team size changes, and 48% estimated that a 25% project delay results in revenue losses of 6%–15% of projected income. This study introduces a rule-based Digital Twin of an Organization (DTO) designed specifically for software development contexts, addressing a gap in existing DTO research, which predominantly targets manufacturing and logistics domains. The system adopts a four- layer architecture comprising an asset modelling layer based on the Asset Administration Shell standard, a knowledge graph layer implemented in a graph database, an inference engine applying explicit rule sets to propagate impact across the organizational graph, and a visualization layer delivering interactive dashboards. Business rules were empirically derived from the same survey data. A logistic regression baseline using stepwise feature selection identified six significant delay predictors: Communication Gap, Client Trust Impact, Disruption Severity, Team Size, Customer Satisfaction Drop, and Project Method achieving an AUC-ROC of 0.72 and 67% accuracy. Eight machine learning models were trained to establish consensus feature importance, with Communication Gap (SHAP=0.46) and Disruption Severity (SHAP=0.35) emerging as the strongest contributors. A Decision Tree trained on consensus features extracted the IF-THEN rules powering the inference engine. Validation against historical project data and expert walkthroughs confirmed the system's accuracy, demonstrating that combining interpretable business rules with graph-based organizational modelling supports evidence-based decision-making in software project management.

Keywords: *Business impact prediction, Digital twin of an organization, Knowledge graph, Rule-Based simulation, Software engineering*

An Integrated Framework for Identifying, Measuring, and Monitoring UX Debt Across the Software Development Lifecycle using Usability Metrics, Accessibility Standards, and Design Consistency Indicators

[1]P.K.L.S. Dayananda* and [2]H.M.K.T. Gunawardhana
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*pklsdayananda@std.appsc.sab.ac.lk

User Experience (UX) debt is the accumulated cost of design shortcuts, unresolved usability problems, accessibility barriers, inconsistent interface patterns, and weak UX governance. Although software teams commonly recognize these problems, they lack an integrated method for identifying, measuring, prioritizing, and monitoring them within agile delivery. This study develops the UX Debt Management Framework (UX-DMF) using a Design Science Research approach. The empirical analysis used 998 anonymized practitioner records and a corresponding thematic coding dataset. Descriptive statistics, Wilson confidence intervals, chi-square tests, Cramér's V, and a Kruskal-Wallis test were used to examine the prevalence and contextual distribution of UX debt. Usability debt was the most frequent category (43.1%), followed by design consistency debt (29.8%), process UX debt (15.1%), and accessibility debt (12.0%). Time pressure was the leading root cause (29.7%). Teams most often relied on hotfixes, partial redesigns, workarounds, or delayed action, while the most frequently reported consequences were productivity loss, user complaints, reduced conversion, brand inconsistency, support demand, negative reviews, and slower onboarding. Debt type was not significantly associated with role, company size, product type, or practitioner experience, indicating that the problem cuts across organizational contexts. The resulting UX- DMF combines a four-dimensional taxonomy, a twelve-indicator scoring rubric, the UX Debt Index Score (UXDIS), the UX Debt Prioritization Score (UXDPS), and an agile lifecycle for recording, prioritizing, repaying, and monitoring debt. Equal dimension weights are used provisionally because a complete expert-derived weighting dataset was not available. The framework provides a practical basis for making UX debt visible and actionable, while future studies should validate the scoring weights and longitudinal effects in live software teams.

Keywords: *Accessibility, Agile software development, Design consistency, UX debt, Usability debt*

A Hybrid Transformer-based Framework for Agile Requirement Prioritization using TinyBERT and CodeBERT Models

[1]A.M.M. Rasmy* and [1]S. Nishankar
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*ammrasmy@std.appsc.sab.ac.lk

Requirement prioritization is a critical activity in Agile software development, enabling teams to rank requirements, user stories, feature requests, and bug reports according to their importance, urgency, and potential impact. However, the increasing volume of unresolved issues in open-source repositories such as GitHub and Jira makes manual prioritization time- consuming, inconsistent, and difficult to scale. This study proposes a hybrid transformer-based deep learning framework that combines TinyBERT and CodeBERT through a concatenation-based feature fusion mechanism to automatically classify software issue priorities into three categories: High, Medium, and Low. TinyBERT is utilized to learn efficient contextual representations from natural language descriptions, while CodeBERT is incorporated to capture software-specific contextual information from technical artifacts such as code fragments, stack traces, and error-related descriptions. The proposed framework was developed and evaluated using a consolidated multi-project dataset containing software issue records collected from open-source repositories, including Cassandra, Firefox, Hadoop, HBase, and Core, together with an additional requirements dataset. Priority labels were standardized across all datasets to enable consistent model training and evaluation. The hybrid model was trained and compared with a standalone TinyBERT baseline using different pooling strategies, class-weighting techniques, and regularization approaches. Model performance was evaluated using Accuracy, Precision, Recall, and macro-averaged F1-score. Experimental results show that the hybrid TinyBERT–CodeBERT model achieved a macro-F1 score of 0.7717, while the standalone TinyBERT model achieved a slightly higher macro-F1 score of 0.7722. Although the hybrid approach did not provide measurable performance improvement on the selected dataset, the results demonstrate that transformer-based models can effectively learn contextual representations from software-related textual data. The findings indicate that the limited presence of code-specific information within issue descriptions may have restricted the additional benefits provided by CodeBERT representations. This study provides empirical insights into hybrid transformer architectures for Agile requirement prioritization and highlights the importance of dataset characteristics in multi-model fusion approaches. Future research can explore advanced fusion mechanisms, larger datasets containing richer technical artifacts, and explainable artificial intelligence techniques to further enhance automated requirement prioritization systems.

Keywords: *Agile software development, CodeBERT, Requirement prioritization, TinyBERT, Transformer models*

An Explainable Geospatial Machine Learning System for House Price Prediction and Real Estate Listing Credibility Assessment in Sri Lanka

[1]V. Akhiliny* and [2]U.P. Kudagamage
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*akhilinyv@gmail.com

The reliance on manual property valuation by professional experts and price comparison by hand is a traditional method in Sri Lankan residential property market, which is not accessible to the majority of buyers and ineffective to the systematic property price inflation in online informal property market. While machine learning techniques provide a potential avenue towards objective automated valuation, current models in Sri Lanka are based on structural attributes of properties, lack geospatial enrichment, lack transparency of their predictions, and lack a way to identify potentially overpriced properties. The current study aims to develop a new Explainable Geospatial Property Valuation System to be applied to the residential property market in Sri Lanka. It is a prediction engine trained using an Optuna Bayesian hyperparameter optimized, LightGBM gradient boosted ensemble with 22 engineered features extracted from 36,202 residential property listings in. Seven geospatial features derived from OpenStreetMap and Geoapify are combined into a new Neighborhood Quality Index (NQI) composite score that measures multi-dimensional neighborhood quality. TreeSHAP is used to break down every price prediction into LKR-denominated feature attributions, understandable by non-technical property buyers. A novel Listing Credibility Score (LCS) framework is proposed that classifies listings based on ML-based fair value comparison and Isolation Forest anomaly detection without the need for manual review by an expert. The effectiveness of the proposed system is validated through empirical testing and it is found that the system can predict the output with R2=0.7946 and the prediction error is reduced four times compared to the latest published Sri Lanka benchmark. The power of the geospatial features has been quantified for the first time in the Sri Lankan market with 39.3% of the total power of the model. The LCS has a classification accuracy of 90.0%, with Cohen's Kappa = 0.80 compared to the expert human evaluation and identifies 7.3% of market listings as potentially overpriced by more than 50% of fair market value. The entire system is deployed as a real-time interactive Streamlit web application which offers price predictions, explanations based on SHAP values, prediction intervals and credibility assessment to Sri Lankan property buyers.

Keywords: *Explainable artificial intelligence, Geospatial intelligence, Listing credibility assessment, Machine learning, Property price prediction*

A Framework for Code Metric Selection in Bug Prediction Based on Developer Perceptions

[1]J.A.G.T. Jayasuriya* and [1]W.V.C. Maduwanthi
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*jagtjayasuriya@std.appsc.sab.ac.lk

Software defects continue to raise maintenance costs and undermine user satisfaction, yet code-metric-based bug prediction remains under-adopted in industry despite decades of academic research. This persistent research–practice gap is widely attributed to low developer awareness, weak workflow integration, and a lack of practitioner-validated guidance on which metrics to use in which context. This study investigates how software developers perceive code metrics for bug prediction and, on that basis, develops and validates a practitioner-informed framework for context-aware metric selection. A mixed-method design was employed across four phases. In Phase 1, a structured questionnaire collected quantitative and qualitative data from software practitioners on their awareness, perceived metric effectiveness, and adoption barriers. In Phase 2, the data were analysed using descriptive statistics, the Kruskal–Wallis H test, and thematic analysis of open-ended responses. The results reveal a substantial awareness deficit, a consistent practitioner preference for process-history metrics such as code churn and prior defect counts over traditional structural metrics such as lines of code, and a dominant adoption barrier of insufficient awareness. Notably, a clear majority of respondents believed that human factors predict bugs better than code metrics alone, and metric perceptions did not differ significantly across experience levels. In Phase 3, these findings were synthesised into a conceptual framework comprising eight evidence-based guidelines, a three-tier metric-priority ranking, and two context maps that tailor metric recommendations to project type and programming language. In Phase 4, the framework was validated by a separate cohort of experienced practitioners, who rated its clarity, the accuracy of its identified barriers, and its overall practicality positively. The study contributes an empirically grounded, context-aware framework that bridges academic metric research and real-world development practice.

Keywords: *Bug prediction, Code metrics, Developer perceptions, Practitioner-Informed framework, Software defect prediction*

AI-Based Test Case Generation from OpenAPI/Swagger Specifications

[1]N.L.A. Samarasekara* and [2]U.A.P. Ishanka
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*nlasamarasekara@std.appsc.sab.ac.lk

Tools like EvoMaster, Schemathesis, and RestTestGen generate API tests automatically, but they only look at the technical structure of an API spec: endpoints, parameter types, schema rules. They completely ignore the plain-English descriptions in the documentation, which often explain important things like business rules, authentication requirements, and hidden constraints. Because of this, the tests they generate are technically correct but shallow: they miss edge cases and security issues that a real QA engineer would catch just by reading the docs. In contrast, newer LLM-based tools like LogiAgent do understand natural language, but they are unreliable: they flag a large number of false positives (33.81% of the time). Consequently, there is a genuine need for something that understands the text and stays accurate. This study proposed a hybrid framework comprising four components: a Schema Parser, an NLP Semantic Engine, a Hybrid Test Logic Composer, and a Test Case Generator. The NLP Semantic Engine, a CodeBERT model fine-tuned on manually labeled API operations, extracts seven categories of semantic information from free-text descriptions, including authentication requirements and explicit and implicit constraints. The Composer integrates this extracted semantic information with structural schema data to construct unified test cases, exported as executable Postman collections and Pytest suites.The NLP engine achieved a 94.3% F1-score on validation. When tested on five real enterprise APIs (Stripe, GitHub, Twilio, PagerDuty, and Shopify) and compared against EvoMaster and Schemathesis, the framework produced more than twice and three times as many test cases, respectively, and 28.6% to 46.6% of those tests came purely from the NLP semantic analysis, meaning neither baseline tool could have generated them at all. It also managed to catch meaningful spec changes across different API versions.

Keywords: *API test case generation, CodeBERT, OpenAPI specification, Semantic constraint extraction*

Bandwidth-Adaptive Gaussian Splatting for Optimized 3D Web Rendering in Resource-Constrained Environments

[1]V. Kajana* and [1]S. Nishankar
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya, Sri Lanka
*vkajana@std.appsc.sab.ac.lk

The challenge of representing and rendering 3D models on the Web in environments with limited bandwidth and limited resources is especially difficult as traditional meth- ods of 3D rendering involve sending all of the scene's high resolution data (slow to transfer, long load times and high GPU memory usage). This thesis presents the design, implementation, and simulation-based evaluation of a Bandwidth-Adaptive Gaussian Splatting (BAGS) framework for efficient 3D rendering on bandwidth-constrained web deployment scenarios. The proposed framework adds an adaptive (Level of Detail) LOD mechanism, an Exponentially Weighted Moving Average (EWMA) bandwidth estimation module and a progressive Gaussian streaming strategy to the existing open- source VkSplat implementation. A prototype was developed in the browser for testing deployment feasibility and quantitative evaluation was performed in Google Colab with the use of Python-based simulations under different necessary bandwidth conditions. The four 3D scenes for evaluation were simulated with five different bandwidths, from 1 Mbps to 100 Mbps. The simulation results demonstrate that the framework could increase the frame rate up to 310.2% and decrease the simulated initial scene loading time up to 88.8% when the bandwidth is low. The EWMA bandwidth estimator had relative estimation errors ranging from 1.4% to 2.2%, and the lower-LOD Gaussian rep- resentations significantly reduced the usage of GPU memory. The results show that the proposed BAGS framework could be an effective bandwidth adaptive rendering strategy and a viable basis for future browser-based 3D Gaussian Splatting systems.Source code is available at https://github.com/kajanav/BAGS

Keywords: *3D gaussian splatting, Bandwidth-Adaptive rendering, Level of detail, WebGL, Web rendering.*

Enhanced Requirement Smell Detection using Transformer-Based Models on Multi-Label Requirement Datasets

[1]L.S.L.S. Lokusiddha* and [1]P.M.A.K. Wijerathne
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*lslslokusiddha@std.appsc.sab.ac.lk

Software requirements written in natural language often contain requirement smells, such as ambiguity, vagueness, subjectivity, incompleteness, and unverifiability, which can reduce software quality and lead to defects. Although automated requirement smell detection has been widely studied, despite individual requirements frequently exhibiting multiple smells simultaneously. This study addresses this limitation by investigating multi-label requirement smell detection using transformer-based models and comparing their performance with traditional machine learning and deep learning approaches. A quantitative experimental methodology was employed, consisting of a pilot study using synthetic dataset and a main study using the publicly available ARTA dataset containing requirement statements labelled with multiple smell categories. Logistic Regression, SVM, and BiLSTM models were evaluated alongside fine-tuned transformer models including DistilBERT, BERT, RoBERT. To address class imbalance, class-weighted learning techniques were applied, and model predictions were further examined using explainability methods. Performance was evaluated using Micro-F1, Macro-F1, Hamming Loss, precision, recall, subset accuracy. The findings highlight the importance of addressing class imbalance in multi-label requirement smell detection and demonstrate that pretrained transformer models can provide effective and interpretable support for automated requirement quality assessment, contributing to improved software requirement engineering practices.

Keywords: *Multi-Label classification, Requirement engineering, Requirement smell detection, Software quality assessment, Transformer models*

Predicting Error Severity Levels in AI-generated Code by Large Language Models

[1]S. Jeyaprashanth*, [1]S. Sweshthika and [2]T. Kayalvizhy
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*sjeyaprashanth@std.appsc.sab.ac.lk

The widespread use of Large Language Models for automated code generation introduces a critical challenge where LLM-generated code frequently contains multi- type errors that require different levels of developer attention. Existing research focuses on error detection rather than systematic severity classification, leaving a significant gap in AI-assisted software development. This study investigates whether machine learning models can effectively classify error severity levels in LLM-generated code across multiple error categories, addressing this gap through a comparative experimental framework. Three datasets were constructed from Python code generated by LLMs and evaluated against HumanEval and MBPP benchmarks via the EvalPlus framework, covering runtime errors with 9 classes, syntax errors with 5 classes, and test failures with 7 classes, with a total of approximately 9,000+ samples collected across all three datasets. Data was pre-processed by combining task prompts, generated code, and error messages as unified inputs, then split using a group-based 70/15/15 train-validation-test strategy to prevent data leakage. Five traditional machine learning models, including Logistic Regression, Random Forest, Linear SVM, Gradient Boosting, and XGBoost were trained using TF-IDF uni-bigram (1,2) features, with Linear SVM achieving the best mean accuracy of 71.83%. Three transformer models, including CodeBERT, RoBERTa, and CodeT5+, were fine-tuned on the same datasets. CodeBERT achieved the highest performance with 93.13% mean accuracy, 91.20% F1 score, 92.01% MCC, and 98.30% ROC-AUC, outperforming both traditional models and general-purpose transformers. Additionally, a code quality optimisation framework was developed, ranking 1,275 verified correct implementations using six weighted metrics including cyclomatic complexity, execution time, and Pylint score. Results confirm that code-specific pre-training provides a substantial advantage for error severity classification. Future work will focus on integrating this framework as a real- time extension for code editors such as Visual Studio Code, providing developers with instant error severity feedback during development.

Keywords: *CodeBERT, Code quality optimisation, Error classification, Large language models, Transformer models*

A Domain-Specific Codebook for Systematic Detection of Hallucinations in LLM-Generated GitHub Actions Workflows

[1]H.P.M.L. Senarathna*, [2]S. Vasanthapriyan and [3]T. Kayalvizhy
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*madhavi.lakmini2000@gmail.com

Continuous Integration pipelines increasingly rely on YAML-based GitHub Actions configurations, which are complex, syntax-sensitive, and error-prone to author manually. Large Language Models hold promise for automating this process, they have the potential to generate outputs that are syntactically correct but semantically flawed, incomplete, or even fabricated, a characteristic known as hallucination. In this study, an empirical analysis is presented of the hallucination behavior of eight LLMs (GPT-4o, GPT-4.1, Qwen2.5, LLaMA3.1-8B, Gemma3-12B, CodeGemma-7B, CodeLlama-7B, and Phi-4-mini) when generating GitHub Actions workflows from natural-language descriptions, using a ground-truth dataset of 1,318 description-YAML pairs. This study proposes a domain-specific annotation codebook to systematically identify and classify hallucinations in GitHub Actions YAML workflows. A heuristic, PyYAML-based classifier was developed and implemented to apply a taxonomy of ten error codes across five categories: Omission, Semantic, Speculative, Cosmetic, and Security. Stratified samples of 100 instances per model, calculated using Cochran's formula for a 95% confidence level and ±10% margin of error, were drawn and validated against consensus labels from five domain-expert annotators, resulting in substantial to almost perfect inter-annotator agreement (Cohen's $\kappa = 0.629$–0.952) across all eight models. Results reveal marked variation in hallucination rates across models, with code-pretrained models such as CodeLlama-7B exhibiting the highest error frequency, and model-specific behavioral patterns such as unsolicited markdown-wrapping observed in Qwen2.5 despite explicit instructions to the contrary. These findings offer a systematic, reproducible framework for evaluating LLM reliability in CI configuration generation and highlight critical considerations for their safe integration into DevOps workflows.

Keywords: *Annotation codebook, CI/CD automation, GitHub actions, Large language models, LLM hallucination*

An Empirical Evaluation of Deep Learning Models for Enhanced and Scalable Bug Detection and Classification in Large-Scale Software Systems

[1]S. Shanthosh* and [1]W.M.L.S. Abeythunga
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*sshanthosh@std.appsc.sab.ac.lk

Software defect prediction is constrained by a structural property of defect data that most classifiers are not designed to handle: defective modules are a small minority of any codebase. A model that maximises overall accuracy on such data can appear successful while detecting almost no real defects. This thesis addresses that gap by empirically evaluating traditional machine learning and deep learning models for defect prediction and bug report classification under explicit imbalance correction. Seven model configurations were trained and evaluated on the JM1 benchmark dataset from the PROMISE repository, comprising 10,885 NASA C- language modules described by 21 Halstead and McCabe complexity metrics with a 4.2:1 class imbalance. Random Forest and Gaussian Naive Bayes served as baselines, each with and without BorderlineSMOTE. A hybrid CNN-LSTM architecture was trained under three loss functions: Binary Cross-Entropy, Focal Loss, and a Class-Weighted Focal Loss proposed in this study, which embeds the observed imbalance ratio directly into the training objective. A DistilBERT transformer was fine-tuned separately for bug report severity classification. The best traditional baseline, Random Forest with BorderlineSMOTE, achieved a recall of 0.451, detecting 190 of 421 defective modules in the held-out test set. The CNN-LSTM trained with Class-Weighted Focal Loss achieved a recall of 0.834, detecting 351 of 421 defective modules, an improvement of 84.7 percent in detection rate, obtained at a measured cost in precision. Ten- fold stratified cross-validation confirmed the stability of the baseline results. DistilBERT achieved perfect classification on the bug report task. The findings establish that data-level and algorithm-level imbalance correction are complementary rather than interchangeable, and that combining them substantially outperforms either strategy applied alone.

Keywords: *Class imbalance, CNN-LSTM, Deep learning, Focal loss, Software defect prediction*

Efficient Human-AI Complaint Routing for Bribery and Corruption Allegations: A CIABOC-Oriented Prototype

[1]K.W.H.M.R.K.S. Elkaduwa* and [2]P.K.D.K. Kaushalya
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*kwhmrkselkaduwa@std.appsc.sab.ac.lk

Public sector complaint handling is important for transparency, accountability, and citizen trust. In bribery and corruption contexts, complaints may include routine inquiries, serious allegations, emotional distress, threats, legal sensitivity, and evidence-related information. Therefore, chatbot- only automation is not suitable for every complaint. This study develops a CIABOC-oriented Human-AI complaint routing prototype that routes simple, low-risk complaints to an AI chatbot while escalating complex, serious, or emotionally sensitive complaints to human officers. Since official CIABOC complaint data were not available, a public Kaggle bribery and corruption complaint dataset was used as a proxy dataset. The complaint complexity component compared several machine learning models using TF-IDF features, and Logistic Regression was selected as the safety-focused model because it achieved the highest complex-class recall and balanced accuracy among the evaluated models. A DistilBERT-based distress detection model and a domain- aware rule layer were integrated to support safer routing. The complete test set of 474 records was reviewed by a lawyer, a business analyst, and an academic lecturer. After review, seven labels were corrected, resulting in 412 simple and 62 complex test records. Using the validated labels, the final hybrid complexity decision achieved 88.40% accuracy and 77.57% balanced accuracy. The routing engine assigned 195 complaints, or 41.14%, to the AI chatbot and 279 complaints, or 58.86%, to human officers. It also identified 44 complaints, or 9.28%, as high priority. Three reviewer- validated complex complaints were routed to the AI chatbot, corresponding to a 4.84% risk rate among complex complaints and a 0.63% overall risky-routing rate. The findings indicate that hybrid Human-AI routing can support early-stage complaint handling while retaining human oversight for complex, serious, and emotionally sensitive cases. However, the prototype was evaluated using proxy data and should not be interpreted as a production-ready CIABOC system.

Keywords: *Automation, DistilBERT, Human-in-the-loop, Natural language processing, Public sector service.*

Hierarchical Ensemble-Based Hybrid Model for Automated Software Requirements Classification

[1]A.K.C. Adhikari* and [2]B.T.G.S. Kumara
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*akcadikari@std.appsc.sab.ac.lk

Manual classification of software requirements into functional and non-functional categories is time-consuming and can be inconsistent when requirements are written in ambiguous natural language. This study developed a hierarchical hybrid classification approach that combines classical machine learning, deep learning, transformer models, and ensemble learning. Seven models - Naive Bayes, Random Forest, Logistic Regression, Support Vector Machine, Convolutional Neural Network, Bidirectional Long Short-Term Memory, and DistilBERT - were first evaluated independently for binary FR/NFR classification and multiclass operation, performance, and security classification. BiLSTM with GloVe embeddings produced the strongest binary baseline, while DistilBERT was selected as the strongest multiclass baseline based on its tied leading accuracy and superior probability-based measurements. These two models formed an initial hierarchical hybrid that achieved 87.54% accuracy. A weighted binary ensemble was subsequently constructed from Naive Bayes, BiLSTM-GloVe, and DistilBERT, and a dynamic confidence-weighted multiclass ensemble was constructed from DistilBERT, SVM, and Logistic Regression. The two ensembles were integrated into a cascaded architecture in which FR requirements terminate at Stage 1 and predicted NFR requirements are routed to Stage 2. On a strict joint holdout of 886 requirements, the final two-ensemble hybrid achieved 95.94% end-to-end accuracy, a 95% bootstrap confidence interval of 94.58%-97.18%, a weighted F1-score of 0.9589, and binary routing accuracy of 0.9752. The findings demonstrate the practical value of combining specialized heterogeneous ensembles within a hierarchical requirements-classification workflow.

Keywords: *Ensemble learning, Functional requirements, Non-functional requirements, Software requirements classification,*

Enhancing Continuous Testing in DevOps and CI/CD Pipelines using Large Language Models

[1]G.C. Sathsiri* and [1]W.V.C. Maduwanthi
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*gcsathsiri@std.appsc.sab.ac.lk

Continuous Testing (CT) is essential to modern DevOps and Continuous Integration/Continuous Delivery (CI/CD), but conventional automation can become a bottleneck because of brittle scripts, high maintenance effort, and slow feedback. This research designs, implements, and evaluates the LLM Continuous Testing Framework (LLM-CTF), an explainable two-agent framework integrated into a CI/CD pipeline. The TestGenAgent generates tests from the live FastAPI interface, while the TestSelectAgent ranks tests by change relevance and executes an adaptive subset. The revised selector preserves known failures, includes deterministic critical-capability tests, protects scores within 0.05 below the 0.80 threshold, and adds eligible medium-relevance tests until the safety limit is reached. Guarded repair is invoked only when an executed selected test fails; candidates must satisfy Abstract Syntax Tree policies, repeated isolated validation, complete selected-batch verification, and rollback checks. The application contains 78 tests, including 13 generated tests that passed and covered 149 of 154 statements in main.py (97%). The original change selected 16 of 78 tests. A credential-free reviewer-evidence campaign addressed the small-sample limitation by replaying 24 controlled change scenarios. All full and selected runs passed; the selected condition reduced executed tests by a mean of 92.58% and achieved an 8.54-times mean wall- time speed-up. Both the full and selected suites detected all 12 injected source-level mutants. Across 600 deterministic score perturbations, a rigid 0.80 cutoff retained fault detection in 44.83% of trials. Eight blind hold-out repair cases produced six twice-validated repairs, one unsafe rejection, and one verified rollback. A populated live server log and a 109/109 secret- free final gate were also included. The results strengthen prototype evidence but remain limited to one purpose-built service, controlled replays rather than production commits, simulated score perturbations rather than repeated hosted-model calls, and offline repair candidates rather than naturally occurring production failures.

Keywords: *CI/CD, Continuous testing, Guarded self-healing, Large language models, Predictive test selection*

Low-Code/No-Code Platforms' Impact on Software Development Productivity

[1]S.S.K.S. Siriwardhana* and [2]H.M.K.T. Gunawardhana
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*sskssiriwardhana@std.appsc.sab.ac.lk

Application low-code/no-code (LCNC) platforms have emerged as a significant paradigm in modern software engineering, enabling organizations to accelerate application development through visual programming environments, reusable components, and automated workflows. The need for quicker software delivery, the necessity for rapid digital transformation, and the lack of qualified software engineers are the main factors driving their growing usage. Concerns about software quality, maintainability, scalability, governance, integration, and long-term sustainability persist despite the fact that several studies show productivity gains linked to LCNC systems. This study examines how LCNC platforms affect software development productivity and assesses how well they adhere to accepted software engineering best practices. Recent peer-reviewed journal articles and conference publications on low- code/no-code development, software productivity, software quality, and organizational adoption were analyzed as part of a systematic literature review (SLR) using a comparative synthesis technique. The results show that LCNC platforms consistently enable rapid application development and digital transformation by reducing development effort, shortening development cycles, encouraging component reuse, and enhancing collaboration between professional developers and business stakeholders. The review also points out persistent problems, such as restricted customization, vendor lock-in, scalability limitations, integration complexity, governance problems, testing restrictions, and the build-up of technological debt in large-scale corporate applications. The data also indicate that software development productivity should be assessed in terms of maintainability, software quality, testing efficacy, governance, and long-term sustainability, in addition to development pace. Thus, rather than replacing traditional programming entirely, the study finds that LCNC platforms work best when used as an adjunct to traditional software development in well- managed hybrid development environments. The results offer businesses, project managers, and software developers' useful advice on how to choose the best software development strategies while striking a balance between quick delivery, high-quality software, and sustainable engineering methods.

Keywords: *Citizen development, Digital transformation, Low-Code/No-Code platforms, Software development productivity, Software engineering, Systematic literature review*

Explainable Semantic Conflict Detection in Software Requirement Engineering using Sentence Transformers

[1]S. Madusha* and [2]S. Adeeba
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*smadusha@std.appsc.sab.ac.lk

Detecting semantic conflicts in natural-language software requirements is a critical and persistent challenge in Requirements Engineering (RE). Existing automated tools rely on keyword matching or simple similarity thresholds that cannot detect conflicts arising from negation, modal strength differences, antonyms, or implicit semantic opposition, and no existing system provides integrated token-level explanations for detected conflicts, limiting practitioner trust and adoption. This study proposes an Explainable Semantic Conflict Detection Framework combining Sentence-BERT (SBERT) embeddings with a Feed-Forward Neural Network (FFN) classifier, integrating Local Interpretable Model-Agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP) to provide interpretable predictions. Each requirement pair is encoded into a novel 3,072-dimensional feature vector concatenating four components: the req1 embedding (e1), req2 embedding (e2), absolute difference ($|e_1 - e_2|$), and element-wise product ($e_1 \odot e_2$). Beyond binary detection, the framework provides a four-layer characterization: conflict type (semantic, functional, redundancy, modal), linguistic feature tag, prediction confidence, and a four-level severity scale (HIGH/MEDIUM/LOW/NONE). The framework was evaluated on a curated dataset of 3,793 requirement pairs from 12 diverse software project domains using a project-aware 70/15/15 stratified split, with no data leakage confirmed across subsets. Among six classifier configurations evaluated, M6 (SBERT+SVM) achieved the highest F1-score (82.54%), while M5 (SBERT+FFN), the primary model selected for explainability integration, achieved 79.07% F1 and 87.29% ROC-AUC, both substantially outperforming the TF-IDF baseline (80.12% F1) and a naïve SBERT cosine-similarity baseline (64.39% F1). M4 (SBERT+RF) achieved the highest recall (92.03%), suited to safety-critical domains where missed conflicts carry high cost. LIME batch analysis revealed antonym conflicts produced the highest detection accuracy (83.3%) while implicit conflicts were the most challenging (62.5%). Severity analysis showed 290 conflict pairs classified as HIGH severity, 9 as MEDIUM, and 2 as LOW, giving requirements engineers actionable prioritization beyond binary labeling. A Streamlit interactive dashboard prototype demonstrates the framework's practical deployability for requirements engineering practitioners without machine learning expertise.

Keywords: *Explainable AI, LIME, Requirements engineering, Sentence-BERT, SHAP*

Comparative Evaluation of Machine Learning and Hybrid Deep Learning Models for Cross-Language Software Code Quality Prediction

[1]A.G.M.A. Nirmantha* and [2]B.T.G.S. Kumara
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*milindanirmantha@gmail.com

Modern open-source software development increasingly involves multiple programming languages, yet existing tools for detecting code quality issues remain predominantly language-specific and rule-based, limiting their applicability in polyglot environments. This study investigates the effectiveness of machine learning and hybrid deep learning approaches for predicting software code quality issues across Java and Python source code, a cross-language capability rarely evaluated under a single, unified, non-circular experimental framework in prior literature. A labelled, cross-language dataset of 5,984 functions was constructed from the CodeSearchNet corpus using independently verified static analysis tools (PMD for Java, radon for Python). Five predictive approaches were developed and systematically compared: a Random Forest classifier operating on handcrafted structural metrics, a fine-tuned CodeBERT transformer, a custom Convolutional Neural Network, and two hybrid architectures fusing structural and semantic representations. All models were benchmarked against an independently configured static analysis tool baseline and evaluated for zero-shot cross- language generalization, with class-imbalance handling, hyperparameter tuning, multi-seed statistical validation, and ensembling applied to ensure the robustness of reported comparisons. Random Forest, combined with SMOTE-based resampling, achieved the strongest performance (F1 = 0.873 on the minority code-smell class), consistently outperforming CodeBERT (F1 = 0.627), a custom CNN, and both hybrid architectures. Notably, structural-semantic fusion improved performance substantially for the lightweight CNN encoder but degraded performance for the transformer encoder, an asymmetry attributed to differing training regimes. All learned models substantially outperformed the static analysis tool baseline (F1 = 0.193), and zero-shot cross-language transfer retained meaningful, though incomplete, predictive capability ($F_1 \approx 0.42$) in both directions. These findings demonstrate that handcrafted structural features processed by a tree-based ensemble can outperform considerably more resource- intensive deep learning approaches for structurally-defined code quality prediction, offering practical guidance for developing lightweight, cross-language, and interpretable software quality assurance tools.

Keywords: *Code smell detection, CodeBERT, Cross-Language prediction, Deep learning, Machine learning*

Automated Software Documentation Generation and Quality Assessment using Large Language Models

[1]N. Pirarththiga*, [2]S. Vasanthapriyan and [3]T. Kayalvizhy
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[3]Department of Computing & Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*npirarththiga@std.appsc.sab.ac.lk

Software documentation is critical for code maintainability and developer productivity, yet manually writing documentation is time-consuming and inconsistent. This research addresses the challenge of automating high-quality software documentation generation using Large Language Models (LLMs). The study aims to develop an intelligent framework called DocGenius that automatically generates, evaluates and maintains software documentation while detecting hallucinations and supporting adaptive updates for evolving code. The dataset was collected from 15 well-known open-source Python repositories on GitHub, yielding 44,118 Python functions. A four-layer input validation framework was applied, resulting in 34,045 valid functions for experimentations. The Claude Haiku LLM was selected after empirically testing multiple models, including Gemini API and Llama 3.1. Documentation generation employed an optimized few- shot prompting strategy using repository-specific examples. Performance evaluation using BLEU (56.37%), ROUGE-1 (81.73%), ROUGE-L (80.89%), and Semantic Similarity (84.61%) demonstrated strong alignment with manually written documentation. Hallucination detection achieved a combined score of 92.20%, with 88.55% of generated documentation confirmed hallucination-free. Human evaluation by three evaluators yielded an overall score of 92.3%, confirming documentation quality. The adaptive documentation component achieved a 97.87% change detection rate. DocGenius demonstrates that LLM-based approaches can significantly reduce documentation effort while maintaining high quality, offering practical application in software development workflows.

Keywords: *Adaptive documentation, Few-Shot prompting, Hallucination detection, Large language model, Software documentation*

A Comparative Analysis of Resource-Efficient Deep Learning Models for ECG Image Classification in Low-Resource Clinical Settings

[1]R.A.M. Wijesooriya* and [2]P.G.P. Kumara
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*ramwijesooriya@std.appsc.sab.ac.lk

Cardiovascular disease remains the leading cause of death worldwide, and the electrocardiogram (ECG) is still the most widely used tool for detecting it. In many low-resource clinics, though, ECGs exist only as printed or scanned images, and the deep learning models capable of reading them tend to be far too large and too computationally heavy for the CPU-only machines such clinics depend on. This study takes aim at that gap with a systematic, deployment- focused comparison of resource-efficient deep learning models for binary (Normal versus Abnormal) ECG image classification. Six convolutional neural network architectures (MobileNetV3-Small, MobileNetV2, EfficientNet-B0, EfficientNet-B1, DenseNet121, and ResNet50) were each trained under two transfer-learning regimes, frozen feature extraction and full fine-tuning, and judged together on macro-F1, model size, and CPU inference time. Post-training INT8 quantization was then applied to the strongest models using two calibration strategies, the MinMax and Histogram observers. Every experiment ran on CPU hardware so that the findings would mirror the intended deployment setting. Fine- tuning beat frozen feature extraction for five of the six architectures, no single model proved universally best (six Pareto-optimal configurations emerged), and the heavy ResNet50 was comprehensively outperformed. Quantization turned out to be highly sensitive to both the calibration strategy and the architecture: the Histogram observer shrank DenseNet121 to 7.63 MB, a 71.9% reduction, while nudging its macro-F1 up to 0.8619, whereas the MobileNet family collapsed under both observers. DenseNet121 with histogram-calibrated INT8 quantization stood out as the deployment-optimal configuration. Taken together, the study offers systematic, multi-objective, CPU-oriented evidence to guide model selection for ECG image screening in resource-constrained clinical settings.

Keywords: *ECG image classification, Low-resource clinical deployment, Model quantization, Resource-efficient deep learning, Transfer learning*

An Empirical Study on Feedback-Driven Automated Program Repair using Large Language Models

[1]J. Vijayaluxshana*, [2]S. Vasanthapriyan and [3]T. Kayalvizhy
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[3]Department of Computing and Information Systems, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*jvluxshana@std.appsc.sab.ac.lk

Software defects remain one of the most persistent and costly sources of failure in software systems, and the growing capability of large language models (LLMs) to generate code has renewed interest in automated program repair (APR). However, LLMs frequently fail to produce a correct patch on their first attempt, particularly for defects whose root cause is not apparent from the buggy code alone. This study presents the design, implementation, and empirical evaluation of a feedback-driven, iterative APR framework that classifies execution failures into three error models - compilation errors, runtime errors, and assertion or logic failures and generates a category specific diagnostic prompt for each before requesting a repair from an LLM, repeating this process until the repaired program produces output matching the expected result or a bounded iteration limit is reached. A supervised XGBoost classifier, trained on a corpus of 10500 labelled defect records assembled from three sources including real-world GitHub issue data, was developed to automatically assign incoming defects to the correct error category, achieving a held-out test accuracy of 95.14 percent. The end-to-end repair pipeline was evaluated on representative defects spanning all three error categories, with all test cases resolved within two repair iterations and under nine seconds of wall-clock time. The framework was implemented as a complete Python pipeline with a web-based interface for interactive use. These findings provide empirical evidence that the structure and specificity of execution feedback, rather than the raw generative capability of the underlying LLM, is a primary determinant of repair success in iterative, LLM-based program repair systems.

Keywords: *Automated program repair, Error classification, Iterative feedback, Software debugging, XGBoost*

Lightweight Validation of LLM-Generated Functional Requirements in Agile Software Development

[1]R.M.N.D. Rathnayake* and [1]R. Nirubikaa
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*rmndrathnayake@std.appsc.sab.ac.lk

Software requirements are increasingly generated using Large Language Models (LLMs), and this has been shown to increase productivity and speed in requirements engineering tasks. The use of automated generation, however, has also been shown to introduce quality problems such as ambiguity, incompleteness, and lack of measurability, and few lightweight validation techniques exist that are practical for Agile software development teams. In this research, we propose and evaluate a lightweight validation framework for LLM-generated functional requirements, applied to a domain that has received little empirical attention: real-world Agile requirement validation using open-weight LLMs. In this domain, the goal of the framework is to help Agile teams detect and prioritize requirement quality issues without relying on formal verification expertise. We use the LLaMA 3.3 70B model, accessed via the Groq API, to generate enhanced versions of 210 functional requirements drawn from the PROMISE NFR Dataset, and we develop a five-rule validation method to assess their quality against the core characteristics of well-formed requirements. We also train and compare six machine learning models and three ensemble strategies to automate this classification, and show that an optimized ensemble of all six models, combining calibrated probability estimates through a support vector machine meta-learner, improves classification accuracy to 97.92%, exceeding the best individual model by 2.09 percentage points. We further find that a four-level prioritization scheme built on these validation outputs aligns closely with human judgment, with 82.9% of human evaluators agreeing with the assigned priority levels, even though agreement on raw quality labels was lower, at 45.8%. Finally, we investigate the practical usefulness of the framework through evaluation by 24 human assessors, who rated its overall usefulness at 3.83 out of 5, suggesting that lightweight, rule-based and ensemble-driven validation can meaningfully support requirement quality assurance in Agile software development without the overhead of formal methods.

Keywords: *Agile software development, Ensemble learning, Large language models, Lightweight validation, Requirements engineering*

Code Review Participation and Its Effect on Team Productivity

[1]Y.S.R.T. Silva* and [2]B.T.G.S. Kumara
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
[2]Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*ysrtsilva@std.appsc.sab.ac.lk

Code review is a central practice in modern pull-request-based development, but the relationship between review participation and measurable software productivity is not fully understood. This study primarily investigates merge success and rework occurrence using 189,699 pull-request records from 69 software systems. The analysis combines review participation, reviewer attributes, pull-request complexity, and project context. Four normalized indices were engineered: Participation Depth Index, Reviewer Expertise Score, Pull Request Complexity Index, and Build Stability Score. Spearman analysis showed that reviewer expertise had the strongest positive association with merge success (rho = 0.2135), while participation depth and pull-request complexity had moderate positive associations with revision count (rho = 0.4563 and 0.4224). Six classifiers - Logistic Regression, Random Forest, XGBoost, CatBoost, a deep multi-layer perceptron, and TabNet - were trained using a leakage-safe 64/16/20 stratified train-validation-test design. The three strongest validation models were combined through weighted soft voting. On the untouched test set, the primary ensembles achieved 86.47% accuracy and 0.9234 ROC-AUC for merge success, and 82.14% accuracy and 0.8951 ROC-AUC for rework occurrence. Build stability was retained as a supplementary exploratory experiment. It achieved 73.90% accuracy and 0.7610 ROC-AUC, but its low recall for unstable builds showed that review-process variables alone are insufficient for reliable build-risk prediction. Future work should add detailed testing, dependency, build-log, configuration, and environment features. The results demonstrate that review participation, reviewer expertise, and pull-request complexity provide useful, outcome-dependent signals for merge and rework prediction.

Keywords: *Code review, Ensemble learning, Machine learning, Pull requests, Software productivity*

A Software Engineering Framework for Transforming User Feedback into Prioritized Maintenance Actions in Mobile Applications

[1]S. Prethigah* and [1]R. Nirubikaa
[1]Department of Software Engineering, Faculty of Computing, Sabaragamuwa University of Sri Lanka
*sprethigah@std.appsc.sab.ac.lk

Insights from user feedback can be crucial for enhancing software quality, prioritizing maintenance efforts and inform evidence-based decisions in software development. Sentiment, quality complaints, particular problems with features, and other unstructured information found in mobile app reviews could be utilized to direct maintenance planning. Current methods, however, predominately enable manual review analysis and/or identification of a single purpose classifier rather than the multi-dimensionality of user feedback matching international quality standards. Thus, the purpose of this research is to fill the above gap by designing a software engineering approach on mobile app review to derive its software quality model (ISO/IEC 25010) into prioritized maintenance actions. In this research study, 16 machine learning models such as classical models, ensemble, gradient boosting, hybrid and deep learning models were analyzed and compared with respect to three research questions on the most violated quality attributes, service gap pattern and ranking of maintenance priority. The study began with a review of the literature on limitations with review classification. Following a systematic stepsharing approach with data integration from nine sources, data preprocessing, ISO normalization, using VADER sentiment analysis, model training, and model testing. In this study, 758,087 app reviews are picked up via three major processes; Python, Scikit-learn, TensorFlow and VADER. From the findings, it was found that the accuracy of the Stacked Ensemble was equal to 86.30% (Macro F1 = 0.863); Usability and Reliability were the most violated quality attributes; and a novel priority formula prioritized Reliability as the highest maintenance priority. The research demonstrates that a mixture of ISO 25010 classification, sentiment analysis, and priority ranking gave industry actionable insights to the mobile application developer.

Keywords: *ISO/IEC 25010, Machine learning, Mobile app reviews, Priority ranking, Software maintenance*

Author Index